

보험사 설계 및 분석

중간 레포트



명지대학교

분산 프로그래밍 1

60221332 박수현

60211647 김민욱

1 프로젝트 개요	3
1.1 프로젝트 개요 (배경 및 목적)	3
1.2 세부 수행 목표	3
1.3 시스템 설계 범위	3
2 요구사항 분석	5
2.1 Functional Requirements	5
2.2 Actor 목록	6
내부 시스템	6
외부 시스템	7
2.3 Use case Diagram	8
보험개발팀	9
심사팀	10
보상처리팀	11
고객	12
2.4 Use case Scenario	13
[CT-01:상품을 설계한다.]	13
[CT-02:보험료를 산출한다.]	13
[CT-03:기초서류를 등록한다.]	15
[CT-04:상품인가를 신청한다.]	15
[CT-05:요율검증을 요청한다.]	16
[CT-06:상품판매를 확정한다.]	18
[UW-01:계약인수를 심사한다.]	18
[UW-02:위험성을 분석한다.]	19
[CL-01:사고를 접수한다.]	20
[CL-02:손해액을 산정한다.]	22
[CL-03:손해를 조사한다.]	23
[CL-04:보험금을 지급한다.]	24
[CS-01:상품가입을 요청한다.]	25
[CS-02:보험상품을 조회한다.]	26
[CS-03:예상보험료를 산출한다.]	26
[CS-04:보험금을 청구한다.]	28
[CS-05:보험계약을 조회한다.]	29
3 설계 모델링	30
3.1 Class Diagram	30
3.2 Class 관계 및 설명	31
Coverage 관계	31
Coverage	32
Deductible	34
Product 관계	35

Product	36
ProductRider	38
ProductCoverage	39
ProductDocument	40
Contract 관계	41
Contract	42
Party	44
SelectedRider	45
SelectedCoverage	46
Insured 관계	47
Insured	48
Car	49
Model	50
DriveScope	51
Claim 관계	52
Claim	53
Accident	55
ClaimDocument	56
DamageInvestigation 관계	58
DamageInvestigation	58
DamageAssessment	60
ClaimPayment	62



1 프로젝트 개요

1.1 프로젝트 개요 (배경 및 목적)

본 프로젝트는 실제 보험사(신동아화재)의 차세대 시스템 RFP(제안요청서) 문서에 대한 이해를 바탕으로 '자동차 보험 상품 관리 시스템'의 요구사항을 분석하고 설계하는 것을 목표로 한다. 목표를 이루기 위해 보험사의 비즈니스 로직(상품 설계, 심사, 보상, 청약까지)을 분석하여 핵심 기능 요구사항을 도출한다. 각 기능에 따른 액터를 추출하여 유스케이스 다이어그램을 작성하고 시스템의 세부적인 동작 흐름을 담은 시나리오를 작성한다. 최종적으로 설계한 내용을 바탕으로 클래스를 추출하여 시스템 구조를 설계한다.

1.2 세부 수행 목표

- **요구사항 분석 및 액터(Actor) 추출:** 자동차 보험 시스템의 흐름에 맞추어 내부 직원(보험기획팀, 심사팀, 보상관리팀), 시스템 이용자(고객), 그리고 데이터 연동이 필요한 외부 연계 기관(보험개발원, 금융감독원, 신용정보원)과 상호작용하는 액터들을 정의한다.
- **유스케이스 다이어그램(Use Case Diagram) 작성:** 추출한 액터들을 중심으로 상품 설계, 계약 심사, 사고 보상, 가입 신청의 각 분야별 핵심 기능 요구사항을 도출하여 다이어그램으로 시각화한다.
- **유스케이스 시나리오(Use Case Scenario) 작성:** 도출된 유스케이스들의 구체적인 동작에 따른 기본 흐름(Basic Flow)과 예외/대안 흐름(Exception/Alternative Flow)을 구체화하여 시스템의 세부적인 시나리오를 정의한다.
- **시스템 구조 설계 및 구현:** 작성된 유스케이스를 바탕으로 클래스를 추출하여 클래스 다이어그램(Class Diagram)을 설계한다.

1.3 시스템 설계 범위

본 프로젝트에서 모델링할 '자동차 보험 상품 관리 시스템'의 기능적 범위는 자동차 사고로 인한 보험금 청구 및 보상 처리를 핵심 도메인으로 하며, 다음 4가지 핵심 영역으로 한정한다.

- **상품 개발 영역:** 보험개발팀이 자동차 보험 상품의 기본 담보, 특약을 선택하고 수익성을 분석하여 상품을 설계한다. 실제로 상품을 판매하기 위해 외부 기관에 효율검증, 인가 신청, 판매확정을 신고하는 과정을 개발한다.

- **계약 심사 영역:** 고객이 상품을 조회하여 예상 보험료를 산출하고 가입을 요청하면 심사팀이 외부 정보를 조회하여 위험성을 분석하고 계약을 인수하는 과정을 개발한다.
- **보상 처리 영역:** 고객이 자동차 사고로 인한 보험금을 청구하면 보상관리팀이 손해 조사를 통해 손해액을 산정하고 합의를 거쳐 보험금을 지급하는 과정을 개발한다.
- **고객 영역:** 고객이 자동차 보험 상품을 조회하고 가입하는 과정과 계약을 조회하여 보상금을 청구하는 과정을 개발한다.



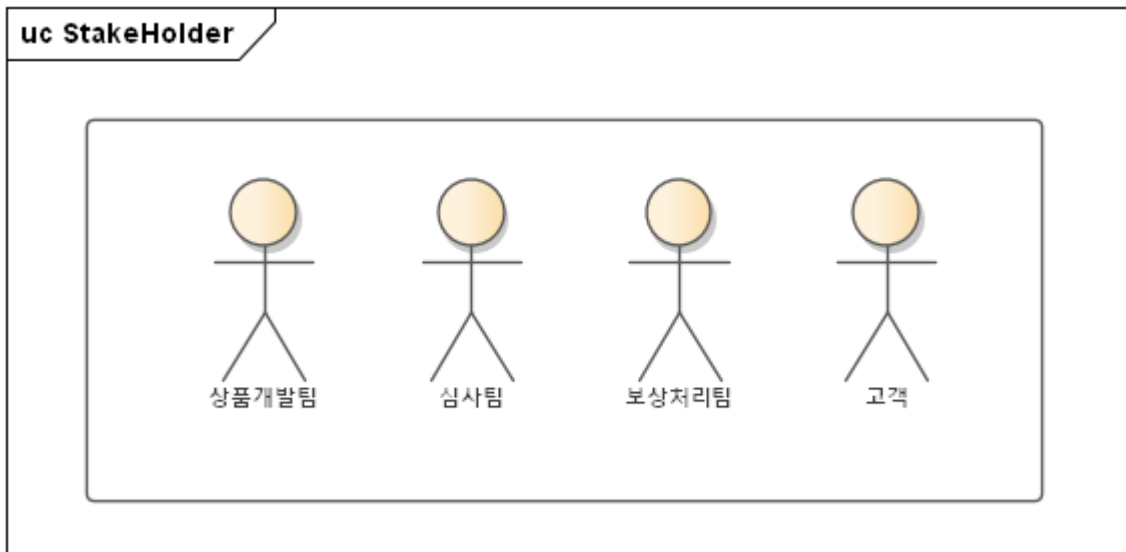
2 요구사항 분석

2.1 Functional Requirements

ID	Name	Description
CT-01	상품을 설계한다.	보험 상품의 보장내용, 요율, 판매기간을 등록하고 관리한다
CT-02	보험료를 산출한다.	보험 상품의 수익성을 분석하고 최종 보험료를 산출한다.
CT-03	기초서류를 등록한다.	보험 상품 설계에 필요한 기초서류를 등록한다.
CT-04	상품인가를 신청한다.	금융감독원에 상품 승인을 신청한다.
CT-05	요율검증을 요청한다.	요율확인서를 받기 위해 보험개발원에 요율 검증을 요청한다.
CT-06	상품판매를 확정한다.	상품을 판매하기 전 금융감독원에 상품 판매를 신고한다.
UW-01	계약인수를 심사한다.	고객의 청약 인수 여부를 결정한다.
UW-02	위험성을 분석한다.	신용정보원에서 고객의 정보를 조회하여 위험성을 분석한다.
CL-01	사고를 접수한다.	고객의 사고 정보를 접수하고 담당 보상직원에게 배당한다.
CL-02	손해액을 산정한다.	손해액을 산정하고 최종 지급 보험금을 결정한다.
CL-03	손해를 조사한다.	사고 현장 조사 결과를 등록하고 면/부책 여부를 판단한다.
CL-04	보험금을 지급한다.	지급 결재 완료 시 고객에게 보험금을 지급한다.
CS-01	상품가입을 요청한다.	고객이 보험 상품에 가입을 신청한다.
CS-02	보험상품을 조회한다.	고객이 보험 상품을 조회한다.
CS-03	예상보험료를 산출한다.	고객이 상품에 가입하기 전 예상 보험료를 계산한다.
CS-04	보험금을 청구한다.	고객이 사고를 당했을 때 보험금을 청구한다.
CS-05	보험계약을 조회한다.	고객의 계약, 보상, 민원 이력을 단일 화면에서 통합 조회한다.

2.2 Actor 목록

내부 시스템



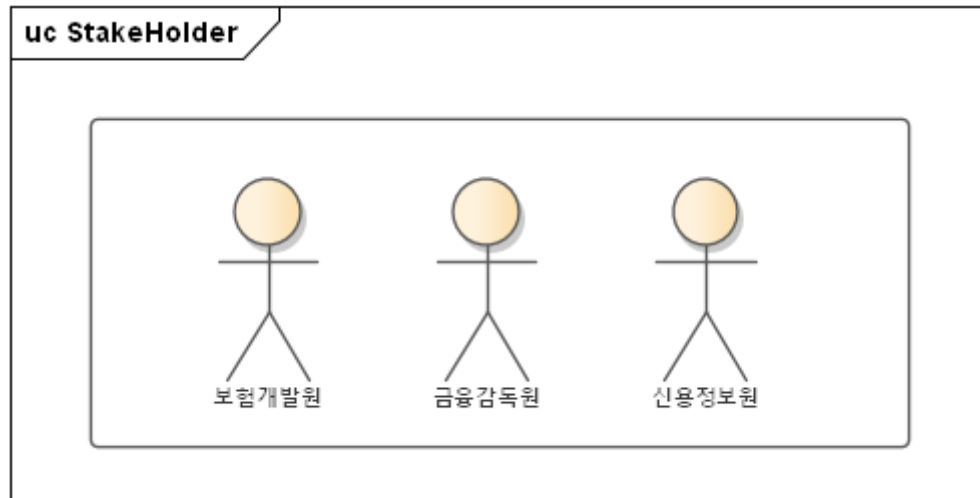
시스템 이용자 (System User)

- 고객 (Customer)
 - 보험사 시스템의 외부 액터이자 핵심 서비스를 직접 이용하는 주체다. 고객은 시스템에 접속하여 보험 상품 조회, 가입 요청, 보험료 산출을 수행한다. 사고 발생 시에는 보험 계약을 조회하고 보상을 청구하는 전체 프로세스의 시작점이 되므로 필수 액터로 정의했다.

내부 직원 (Internal Employee)

- 상품개발팀 (또는 보험기획팀)
 - 신규 보험 상품의 설계부터 판매 확정까지의 전 과정을 주도하는 액터임. 상품의 담보/특약 설계, 기초 서류 등록, 수익성 분석 및 외부 기관(보험개발원, 금융감독원)과의 인가/검증 프로세스를 시스템 내에서 총괄하므로 별도 액터로 도출함.
- 심사팀 (Underwriting Team)
 - 고객의 가입 요청에 대해 계약 인수 여부를 결정하는 전문 부서임. 외부 기관으로부터 고객의 이력을 조회하여 위험성과 손해율을 분석하고 최종 승인 및 거절을 심사하는 고유의 역할을 담당하므로 독립적인 액터로 식별함.
- 보상처리팀 (Compensation Team)
 - 고객의 사고 접수 후 보상 업무 전반을 시스템에서 처리하는 부서임. 사고 조사, 과실 비율 파악, 손해액 산정 및 계약 조건(면/부책) 검토 후 최종적으로 결제망을 통해 보험금을 지급하는 핵심 로직을 담당하므로 필수 액터로 지정함.

외부 시스템

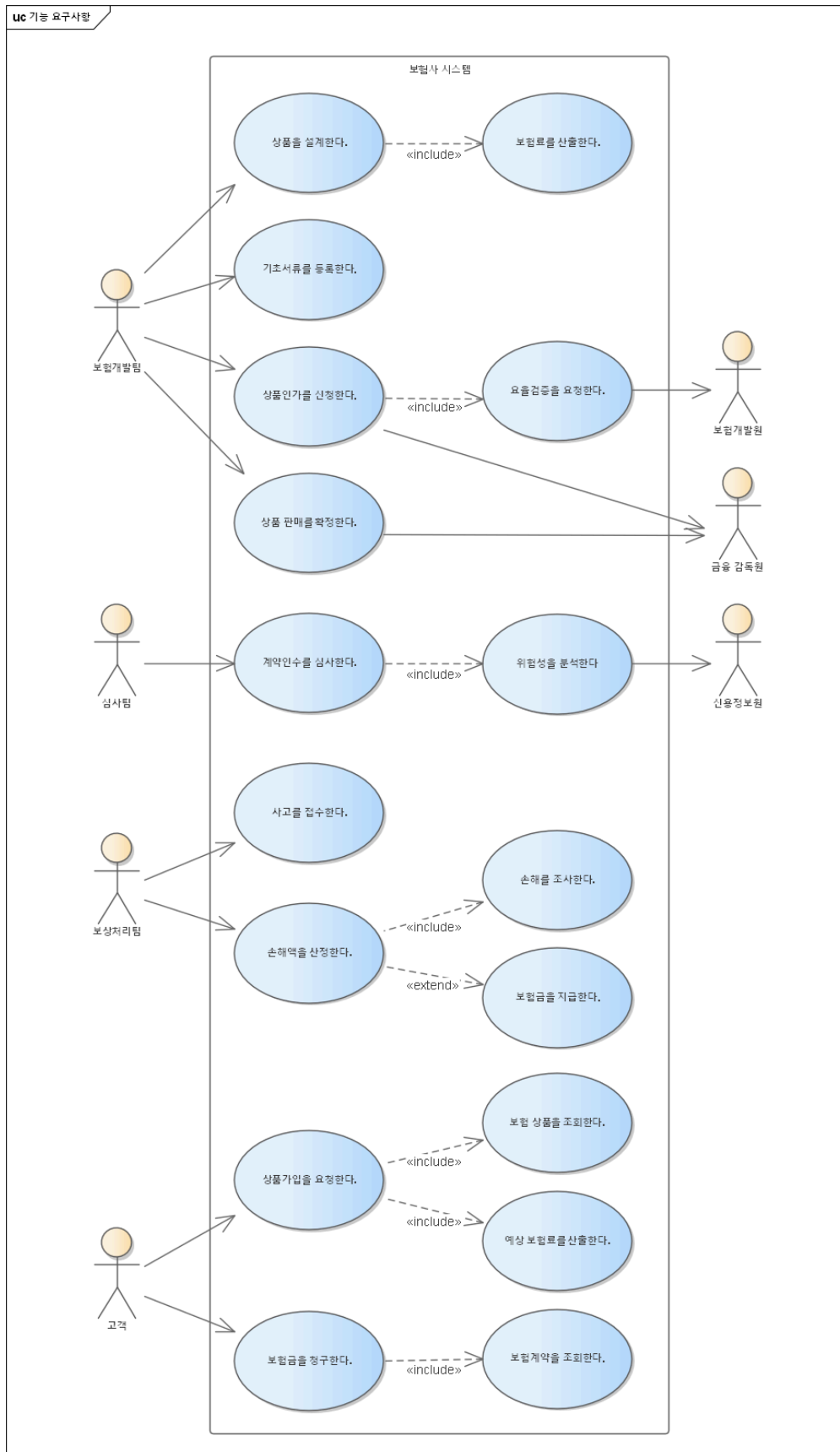


외부 연계 기관 (External Linked Institution)

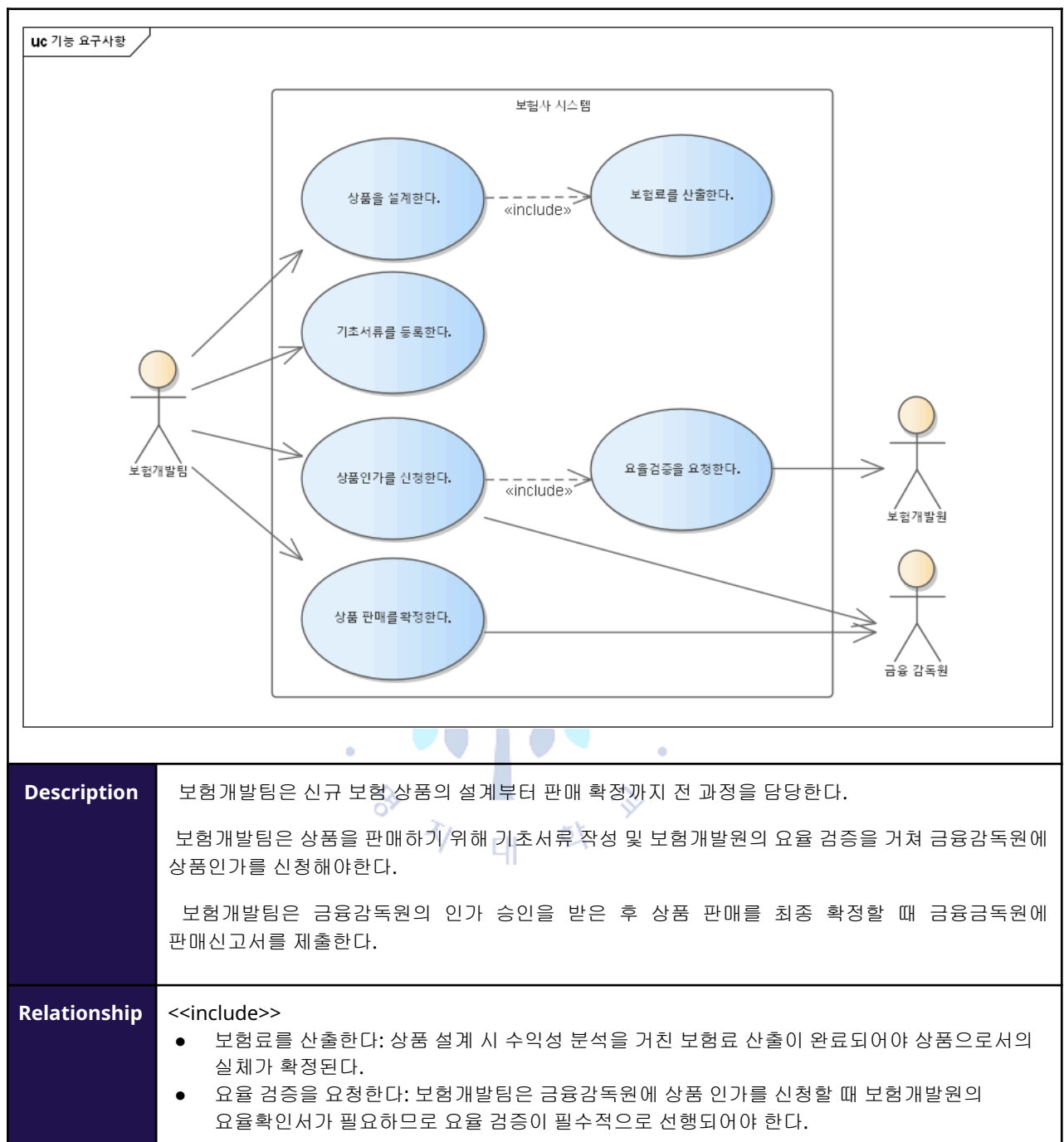
보험 비즈니스는 내부 시스템만으로 완결될 수 없다. 법적 규제 준수 및 정밀한 위험 분석을 위해 외부 시스템과의 데이터 연동이 필수적이므로 3개의 외부 액터를 도출했다.

- 보험개발원 (Insurance Development Institute)
 - 상품개발팀이 설계한 신규 보험 상품의 효율 및 수익성이 적절한지 객관적으로 검증하기 위해 시스템 상에서 '효율 검증'을 요청받고 회신하는 역할을 수행하므로 액터로 추출했다.
- 금융감독원 (Financial Supervisory Service)
 - 설계된 상품에 대해 '상품 인가'를 승인하고 최종적인 '상품 판매 확정' 신고를 처리하는 국가 규제 기관이다. 법적 인허가 절차를 시스템에 반영하기 위해 도출했다.
- 신용정보원 (Korea Credit Information Services)
 - 심사팀이 고객의 계약 인수를 심사할 때 고객의 신용등급, 과거 사고 이력, 보험사기 이력처럼 내부 데이터만으로는 파악할 수 없는 외부 이력을 제공하여 '위험성 분석'의 근거 데이터를 연동해 주는 필수 기관이다..

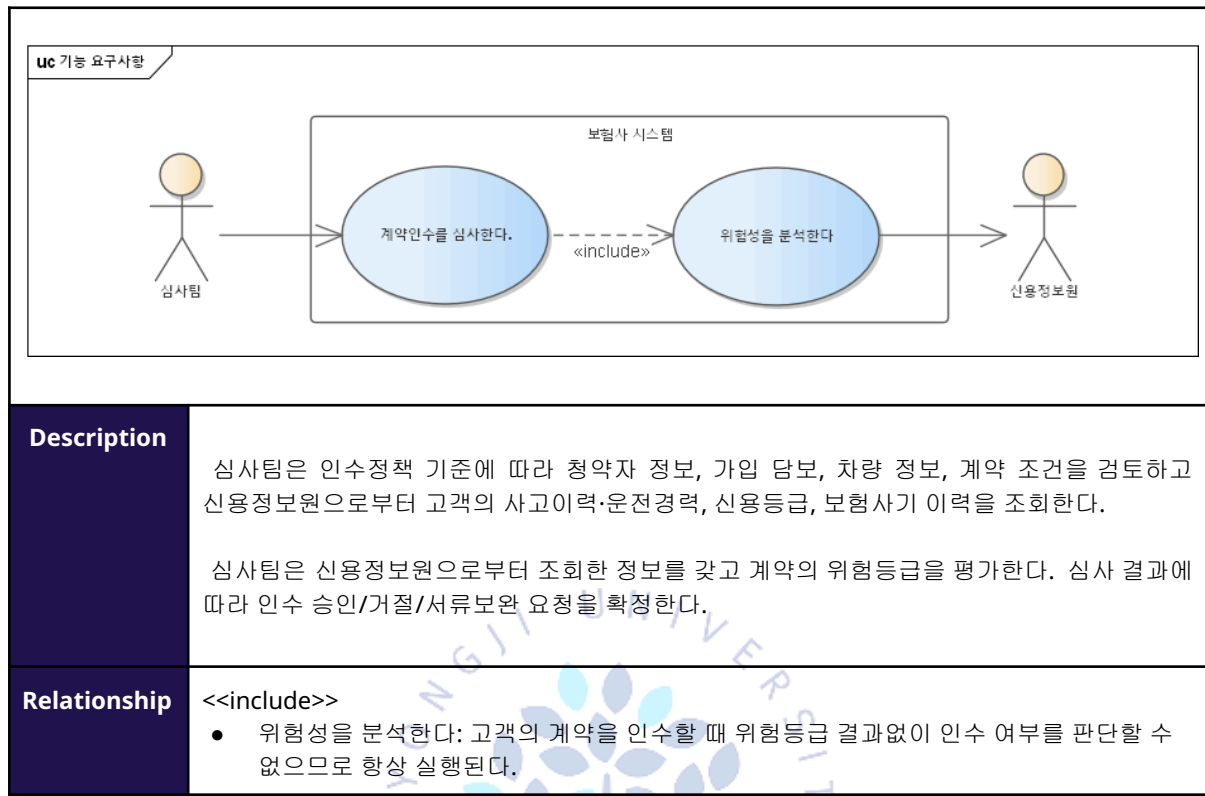
2.3 Use case Diagram



보험개발팀



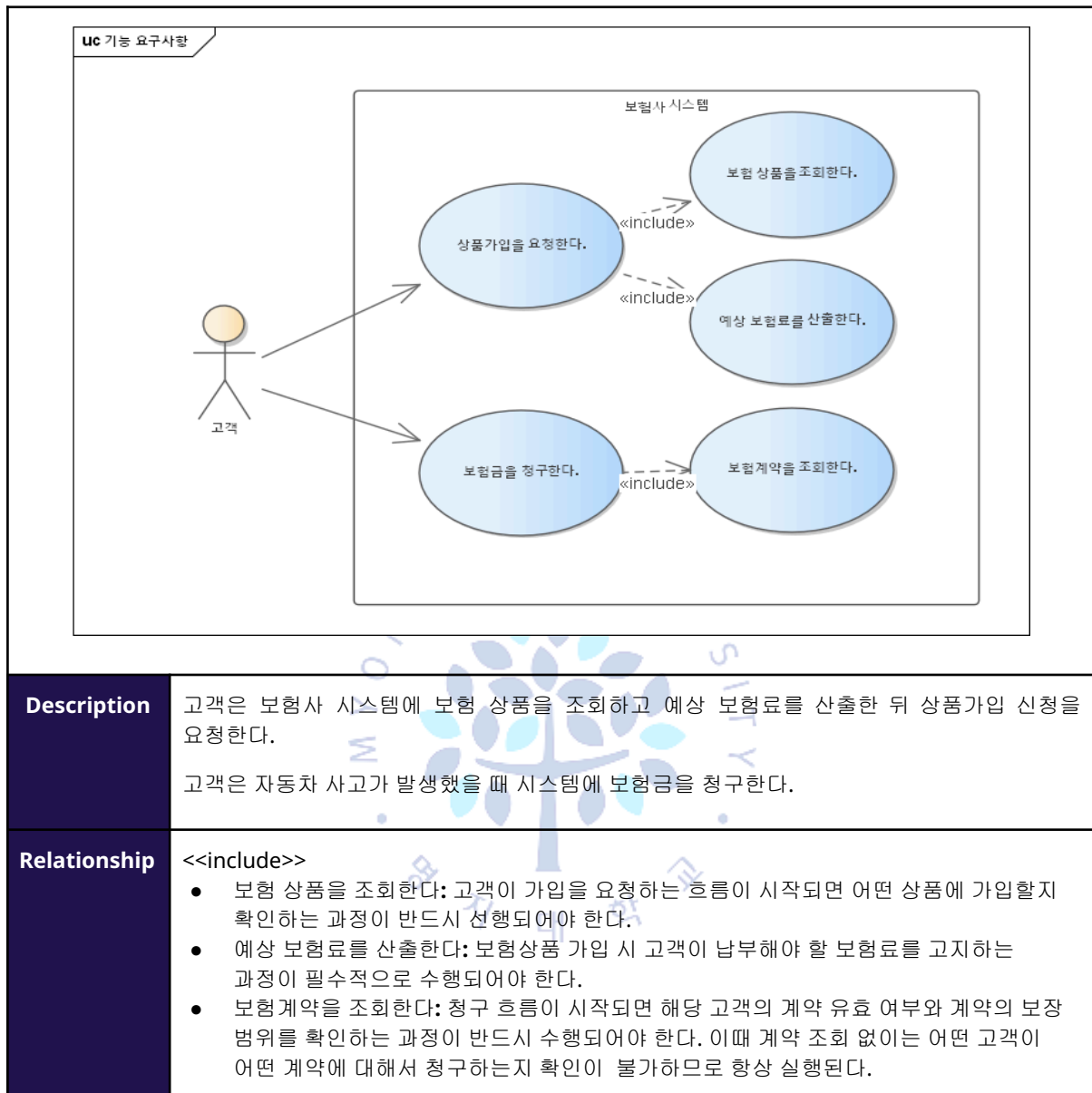
심사팀



보상처리팀

	uc 기능 요구사항 보험사시스템 사고를 접수한다. 손해액을 산정한다. 손해를 조사한다. 보험금을 지급한다. «include» «extend»
Description	보상처리팀은 고객이 제출한 사고 접수 내용을 바탕으로 사고 정보를 시스템에 등록하고 사고 발생 여부와 청구 내용을 확인한다. 보상처리팀은 접수된 사고에 대해 손해 내용을 조사하고 사고 원인과 피해 규모를 확인하여 손해액을 산정한다. 보상처리팀은 산정된 손해액과 계약 조건, 면책사유 여부를 검토하여 보험금 지급 대상 여부를 판단한다. 지급 대상인 경우 보험금을 지급하고 지급 대상이 아닌 경우 지급하지 않는다.
Relationship	<<include>> - 손해를 조사한다: 손해액을 산정하는 흐름이 시작되면 사고 원인과 피해 규모를 파악하기 위한 손해 조사가 반드시 선행되어야 한다. 조사 없이는 손해액의 근거를 확인할 수 없으므로 항상 실행된다. <<extend>> - 보험금을 지급한다: 손해액 산정이 완료된 이후 지급 조건(면책사유 없음, 계약 유효 등)을 충족하는 경우에만 보험금 지급이 이루어진다. 조건을 충족하지 못하면 지급이 발생하지 않으므로 <<extend>>로 표현했다.

고객



2.4 Use case Scenario

[CT-01:상품을 설계한다.]

Actors	상품개발팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 [상품관리] 탭을 클릭한 후 하위 메뉴인 [신규 상품 등록] 메뉴를 선택한다.
2	시스템은 신규 상품을 등록하는 폼(상품명, 상품코드, 보험종목, 판매시작일, 판매종료일, 가입대상, 설명란)과 [다음] 버튼을 출력한다.
3	사용자는 상품명: MZ 세대 다이렉트 차보험, 상품코드: CAR-2026-MZ, 보험종류: 개인용자동차보험, 판매 시작일: 2026-05-01, 판매 종료일: 2027-04-30, 가입대상: 만 20세 이상 39세 이하 운전자를 입력하고 [다음] 버튼을 누른다. [E1]
4	시스템은 담보 목록(대인배상 I, 대인배상 II, 대물배상, 자동차상해, 자기차량손해, 무보험차상해), [가입 옵션선택] 버튼, [다음] 버튼을 출력한다.
5	사용자는 담보 선택 화면에서 상품에 적용할 담보(대인배상 I, 대인배상 II, 대물배상)를 클릭한 뒤 [가입 옵션 선택] 버튼을 누르고 상품에 적용할 가입옵션(대인배상 I - 기본 옵션[V], 대인배상 II - 한도5억[V], 무한[V] 대물배상 - 기본옵션[V])을 선택하고 [다음] 버튼을 누른다. [A1]
6	시스템은 특약 목록(블랙박스할인특약[], 마일리지할인특약[])과 [다음] 버튼을 출력한다.
7	사용자는 추가할 특약(블랙박스할인특약[V])을 선택하고 [다음] 버튼을 누른다.
8	시스템은 설계된 상품 요약 정보(상품명, 판매 기간, 가입대상, 선택된 담보와 가입 옵션, 선택된 특약)와 [보험료 산출] 버튼을 출력한다.
9	사용자는 [보험료 산출] 버튼을 누른다. (<<include>> CT-02: 보험료를 산출한다.)
10	시스템은 분석 결과가 반영된 최종 산출 보험료(기본형 1,150,000원)와 준비금 적립 필요액(460,000원)과 [상품확정] 버튼을 화면에 출력한다.
11	사용자는 [상품 확정] 버튼을 클릭한다.
12	시스템은 "상품 설계가 완료되었습니다." 팝업을 출력한다.
Alternative Flow	
Step	Interaction
A1	필수 담보가 선택되지 않은 경우 1. 시스템은 "대인배상 I은 자동차보험의 필수 담보입니다. 담보 리스트에 추가해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 대인배상 I을 담보 리스트에 추가하고 Basic Flow 5번을 재수행한다.
Exception Flow	
Step	Interaction
E1	상품코드가 기존 상품과 중복된 경우 1. 시스템은 "이미 사용 중인 상품코드입니다"라는 메시지를 출력한다.

[CT-02:보험료를 산출한다.]

Actors	상품개발팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 산출된 보험료 요약과 수익성 분석 입력 폼(목표 판매 건수, 분석 대상 기간)과 [다음] 버튼을 출력한다. (CT-01에 의해 include)
2	사용자는 목표 판매 건수(5,000건)와 분석 대상 기간(2026-05-01 ~ 2027-04-30)을 입력하고 [다음] 버튼을 누른다. [A1]
3	시스템은 보험개발원 참조위험률 기반의 유사 상품의 과거 손해율(대인: 78.5%, 대물: 68.2%)을 참조값과 예상 손해율, 목표사업비율 입력란과 [보험료 산출] 버튼을 출력한다.
4	사용자는 예상 손해율(72.0%)과 목표 사업비율(영업비 15%, 관리비 10%)을 입력하고 [보험료 산출] 버튼을 클릭한다.
5	시스템은 담보별 산출 내역(순보험료: 828,000원, 부가보험료: 322,000원)을 합산한 최종 기본 보험료(1,150,000원)와 담보별 요율 비중(대인 35%, 대물 40%, 기타 25%)과 [수익성 시뮬레이션] 버튼을 출력한다. [E1]
6	사용자는 [수익성 시뮬레이션] 버튼을 클릭한다.
7	시스템은 예상 현금 흐름 분석 결과(총 수입보험료: 5,750,000,000원, 예상 지급보험금: 4,140,000,000원, 예상 사업비: 1,437,500,000원)와 계리적 수익성 평가지표인 NPV(순현재가치: 125,000,000원), IRR(내부수익률: 8.5%), 합산비율(Combined Ratio: 97.0%) 및 손익분기점(BEP: 4,200건 판매 시점)과 [분석 결과 반영] 버튼을 출력한다. [E1]
8	사용자는 수익성 분석 결과가 내부 가이드라인을 충족하는지 최종 확인하고 [분석 결과 반영] 버튼을 클릭한다.
9	시스템은 "최종 확정 보험료는 1,150,000원, 법정 준비금 적립 필요액은 460,000원입니다." 안내메시지를 출력한다. (CT-01의 Basic Flow 10번으로 돌아간다.)
Alternative Flow	
Step	Interaction
A1	분석 대상 기간이 판매기간을 초과할 경우 1. 시스템은 "분석 대상 기간은 상품 판매 기간(2027-04-30 종료)을 초과할 수 없습니다."라는 메시지를 출력한다.
Exception Flow	
Step	Interaction
E1	시뮬레이션을 이용할 수 없는 경우 1. 시스템은 "현재 수익성 시뮬레이션을 이용할 수 없습니다" 안내 메시지를 출력한다.

[CT-03:기초서류를 등록한다.]

Actors	상품개발팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 [상품관리] 탭을 클릭한 후 하위 메뉴인 [상품목록]을 선택한다.
2	시스템은 등록된 상품 리스트(MZ세대 다이렉트 영업용자동차보험, MZ세대 다이렉트 개인용 자동차 보험)를 출력한다.
3	사용자는 'MZ 세대 다이렉트 개인용자동차보험'을 선택한다.
4	시스템은 선택된 상품의 상세 정보(보험종목:개인용자동차보험, 판매시작일: 2026-05-01, 판매 종료일: 2027-04-30, 가입대상: 만 20세 이상 39세 이하 운전자, 현재상태: 설계완료)와 [수정] 버튼, [인가 관리], [판매관리] 버튼을 출력한다.
5	사용자는 [인가 관리] 버튼을 클릭한다.
6	시스템은 상품의 인허가 진행 현황(설계중), 기초서류목록(등록된 문서 없음)과 [서류등록], [요율 검증], [인가 신청], [인가 완료] 버튼을 출력한다.
7	사용자는 [서류등록] 버튼을 누른다.
8	시스템은 각 서류별 업로드 폼(서류종류, 제목, 파일업로드)과 [저장] 버튼을 출력한다.
9	사용자는 준비된 기초서류 파일(사업방법서.pdf, 보험약관.docx, 산출방법서.xlsx)을 각 항목의 파일 선택창을 통해 업로드하고 [저장] 버튼을 누른다. [A1] [E1]
10	시스템은 "파일이 성공적으로 저장되었습니다." 팝업창을 출력한다.
Alternative Flow	
Step	Interaction
A1	업로드 파일 형식이 xlsx, docx, hwp, pdf 가 아닌 경우 1. 시스템은 "파일 형식이 올바르지 않습니다. xlsx, docx, hwp, pdf 형식의 파일만 업로드할 수 있습니다."라는 경고 메시지를 출력한다. 2. 사용자는 올바른 형식의 파일을 선택하여 Basic Flow 7번을 재수행한다.
Exception Flow	
Step	Interaction
E1	파일 업로드에 실패한 경우 1. 시스템은 "파일 업로드에 실패했습니다. 다시 시도해주세요" 팝업창을 출력한다.

[CT-04:상품인가를 신청한다.]

Actors	상품개발팀, 금융감독원
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 [상품관리] 탭을 클릭한 후 하위 메뉴인 [상품목록]을 선택한다.
2	시스템은 등록된 상품 리스트(MZ세대 다이렉트 영업용자동차보험, MZ세대 다이렉트 개인용자동차보험)를 출력한다.
3	사용자는 'MZ 세대 다이렉트 개인용자동차보험'을 선택한다.
4	시스템은 선택된 상품의 상세 정보(보험종목:개인용자동차보험, 판매시작일: 2026-05-01, 판매 종료일: 2027-04-30, 가입대상: 만 20세 이상 39세 이하 운전자, 현재상태: 설계완료)와 [수정] 버튼, [인가 관리], [판매관리] 버튼을 출력한다.
5	사용자는 [인가 관리] 버튼을 클릭한다.
6	시스템은 상품의 인허가 진행 현황(설계중), 기초서류목록(보험약관, 사업방법서, 보험료 및 책임준비금 산출서)과 [서류등록], [요율 검증], [인가 신청], [인가 완료] 버튼을 출력한다.
7	사용자는 [요율 검증] 버튼을 누른다. (<<include>> CT-05: 요율 검증을 요청한다.)
8	시스템은 보험상품 인가신청서를 등록할 수 있는 입력란과 [저장] 버튼을 출력한다.
9	사용자는 보험상품 인가신청서를 등록한 후 [저장]버튼을 누르고 [인가 신청] 버튼을 클릭한다.
10	시스템은 "인가 신청이 완료되었습니다. 15일 이내에 확인 결과가 통보됩니다."라는 메시지를 출력한다. [E1] [E2]
11	사용자는 추후 금융감독원으로부터 회신된 상품 적정성 심사 결과 보고서를 확인하고 [인가 완료] 버튼을 클릭한다. [A1] [A2]
12	시스템은 "해당 상품의 인가가 최종 승인되었습니다." 팝업창을 출력한다.
Alternative Flow	
Step	Interaction
A1	금융감독원 심사 결과 부적합 판정을 받은 경우 1. 시스템은 Basic Flow 6번으로 이동하여 서류 등록을 다시 수행한다.
A2	금융감독원 심사 결과가 '보완 요청'인 경우 1. 시스템은 Basic Flow 6번으로 이동하여 보완할 서류를 등록한다.
Exception Flow	
Step	Interaction
E1	필수 제출 서류 중 '보험개발원 확인서'가 누락된 경우 1. 시스템은 "보험개발원의 확인서가 업로드되지 않았습니다. 해당 서류는 인가 요청의 필수 항목입니다."라는 경고창을 띄운다.
E2	금융감독원에 서류제출을 실패한 경우 1. 시스템은 "금융감독원으로 서류 제출을 실패했습니다. 다시 시도해주세요."를 출력한다.

[CT-05:요율검증을 요청한다.]

Actors	상품개발팀, 상품개발원
Preconditions	CT-04: 상품인가를 요청한다
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 현재 보험상품의 상태(설계중)와 보험개발원 요율검증 요청에 필요한 제출 서류 목록과 입력란과 [저장]버튼을 출력한다. (CT-04: 상품인가를 요청한다.에 의해 include)
2	사용자는 요율 산출 근거서, 담보별 기준 순보험료 산출표를 파일로 첨부하고 [저장]버튼을 누른다.
3	시스템은 "서류 첨부이 완료되었습니다." 팝업창을 출력하고 [보험개발원 제출]버튼을 출력한다.
4	사용자는 [보험개발원 제출] 버튼을 클릭한다.
5	시스템은 "보험개발원으로 서류가 제출되었습니다. 최대 15일이 소요될 수 있습니다."라고 출력한다. [E1]
6	사용자는 추후 보험개발원에서 요율확인서를 수령하여 상품 상세 페이지의 [인가 관리] 메뉴의 [서류 등록] 탭을 열어 요율확인서를 업로드하고 [저장] 버튼을 클릭한다. [A1]
7	시스템은 "요율확인서가 등록되었습니다. 인가신청서를 등록하시겠습니까?"라는 메시지와 함께 [예]/[아니오] 버튼을 출력한다.
8	사용자는 [예] 버튼을 누른다.
9	시스템은 "인가신청서를 등록화면으로 이동합니다." 팝업창을 출력한다. (CT-04: 상품인가를 요청한다.의 Basic Flow 8번으로 넘어간다.)
Alternative Flow	
Step	Interaction
A1	보험개발원 검증 결과가 '보완요청'인 경우 1. 시스템은 보험개발원으로부터 수신한 보완 사유(산출 정확성 미달)를 화면에 출력한다. 2. 사용자는 보완 사유를 확인하고 Basic Flow 2번을 재수행한다.
Exception Flow	
Step	Interaction
E1	보험개발원 시스템 전송이 실패한 경우 1. 시스템은 "보험개발원 시스템 연결에 실패하였습니다. 잠시 후 다시 시도해 주세요."라는 경고 메시지를 출력한다.

[CT-06:상품판매를 확정한다.]

Actors	상품개발팀, 금융감독원
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 【상품관리】 탭을 클릭한 후 하위 메뉴인 【상품목록】 을 선택한다.
2	시스템은 등록된 상품 리스트(MZ세대 다이렉트 영업용자동차보험, MZ세대 다이렉트 개인용자동차보험)를 출력한다.
3	사용자는 'MZ 세대 다이렉트 개인용자동차보험'을 선택한다.
4	시스템은 선택된 상품의 상세 정보(보험종목:개인용자동차보험, 판매시작일: 2026-05-01, 판매 종료일: 2027-04-30, 가입대상: 만 20세 이상 39세 이하 운전자, 현재상태: 설계완료)와 【수정】 버튼, 【인가 관리】 , 【판매관리】 버튼을 출력한다.
5	사용자는 【판매관리】 버튼을 클릭한다.
6	시스템은 상품의 판매 현황(판매 전), 【판매신청】 , 【판매개시】 , 【판매중단】 버튼을 출력한다.
7	사용자는 【판매신청】 버튼을 누른다.
8	시스템은 판매 확정 신청 서류 업로드 화면과 【금융감독원 제출】 버튼을 출력한다.
9	사용자는 상품 신고서, 수익성 분석 보고서, 공시자료를 업로드하고 【금융감독원 제출】 버튼을 클릭한다.
10	시스템은 “금융감독원으로 서류를 제출하였습니다.”를 출력한다. [E1] [E2]
11	사용자는 추후 금융감독원으로부터 판매 승인이 되면 【판매관리】 의 【판매개시】 버튼을 누른다.[A1] [A2]
Alternative Flow	
Step	Interaction
A1	금융감독원 판매 확정 승인이 거부된 경우 1. 사용자는 거부 사유를 확인하고 Basic Flow 4번으로 이동한다.
A2	금융감독원 판매 확정 승인이 보완 요청인 경우 1. 사용자는 보완 서류 목록을 확인하고 Basic Flow 4번으로 이동한다.
Exception Flow	
Step	Interaction
E1	필수 제출 서류가 누락된 채로 제출을 시도한 경우 1. 시스템은 "필수 서류가 누락되었습니다."라는 경고 메시지를 출력하고 누락된 서류의 업로드 칸을 강조하여 출력한다. 2. 사용자는 누락된 서류를 업로드하고 Basic Flow 5번을 재수행한다.
E2	금융감독원으로 서류전송이 실패한 경우 1. 시스템은 “금융감독원으로 서류 제출을 실패했습니다 다시 시도해주세요.”라고 오류 메시지를 출력한다. 2. 사용자는 Basic Flow 8번을 재수행한다.

[UW-01:계약인수를 심사한다.]

Actors	심사팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 [계약관리] 탭을 클릭한 후 하위 메뉴인 [계약목록] 메뉴를 선택한다.
2	시스템은 심사 대기 중인 청약 목록(청약번호: 20260401-0001, 청약자명: 박수현, 상품명: MZ 세대 다이렉트 차보험, 보험료: 2,907,200원, 청약일자: 2026-04-01, 상태: 심사대기중)을 출력한다.
3	사용자는 목록에서 심사할 청약 건(20260401-0001)을 클릭한다.
4	시스템은 청약 상세 정보(성명: 박수현, 주민번호: 020101-3*****, 주소: 서울시 강남구, 차량번호: 64마0866, 차대번호: KMHCT41DBLU123, 가입담보: 대인1/2, 대물 5억, 자상 1억, 무보험 2억, 자차 가입)와 [위험성 분석] 버튼을 출력한다.
5	사용자는 [위험성 분석] 버튼을 클릭한다. (<<include>> UW-02: 위험성을 분석한다.)
6	시스템은 분석 완료된 외부 기관 정보와 산출된 위험분석 보고서(위험 등급: 3등급[조금 위험], 사고점수: 1.5점, 법규위반점수: 0점) 및 인수 지침(심사 가이드: 사고이력 존재하나 할증 범위 내)과 최종 심사 의견란과 [저장] 버튼을 출력한다.
7	사용자는 산출된 위험 결과와 청약자의 정보(직업:대학생, 연령: 만 24세)를 대조하여 최종 심사 의견을 입력란에 기재하고 [저장]버튼을 누른다.
8	시스템은 최종 결과 화면(최종 산출 보험료 요약: 기본 보험료(2,794,010원) + 위험 할증(+5%: 139,700원) = 합계 2,933,710원, 심사 이력 정보: 심사역(김민욱), 심사 일시: 2026-04-25 22:15)과 [인수 승인], [인수 거절], [서류보완 요청] 버튼을 출력한다.
9	사용자는 [인수 승인] 버튼을 클릭한다. [A1] [A2]
10	시스템은 "계약번호(CN-2026-9981)의 인수가 승인되었습니다." 팝업창을 출력한다. [E1]
Alternative Flow	
Step	Interaction
A1	위험성이 높아 인수를 거절할 경우 1. 사용자는 [인수 거절] 버튼을 클릭하고 인수거절 사유를 입력하고 저장한다. 2. 시스템은 "인수 거절과 함께 거절 사유를 전송했습니다."를 출력한다.
A2	위험성이 높아 추가 서류가 필요한 경우 1. 사용자는 [서류보완 요청] 버튼을 클릭하고 보완할 서류 목록을 클릭하고 저장한다. 2. 시스템은 "서류 보완을 요청했습니다."를 출력한다.
Exception Flow	
Step	Interaction
E1	시스템 내부 오류로 인수 승인이 불가한 경우 1. 시스템은 "현재 시스템 내부 오류로 인해 승인할 수 없습니다. 다시 시도해주세요"메시지를 출력한다.

[UW-02:위험성을 분석한다.]

Actors	심사팀, 신용정보원
Preconditions	UW-01: 계약인수를 심사한다
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 신용 정보 조회 화면(조회 대상자 정보(박수현, 020101-3*****, 64마0866)와 조회 항목, [조회] 버튼을 출력한다. (UW-01에 의해 include)
2	사용자는 조회 항목(최근 3년 사고이력, 운전경력, 신용등급, 보험사기 의심이력)을 선택하고 [조회] 버튼을 클릭한다. [A1] [E1]
3	시스템은 청약자의 최근 3년간 사고이력(2025-08-01: 타차가해 300만원, 대인처리 1건), 운전경력(보험 가입 경력 2년), 신용등급(NICE: 5등급), 보험사기 이력(해당 없음)과 [위험등급 산출] 버튼을 화면에 출력한다.
4	사용자는 청약자의 정보를 확인하고 [위험등급 산출] 버튼을 클릭한다.
5	시스템은 최종 위험 점수 1.6점, 위험등급 '3등급', 보험료 할증율(+5%)과 [상세보기] 버튼을 출력한다.
6	사용자는 할증된 사유를 검증하기 위해 [상세보기] 버튼을 클릭한다.
7	시스템은 위험등급 산출 상세 내역 팝업창을 출력하고 항목별 감점 요인(사고 건수: 1건-1.2점, 운전 경력: 2년-0.3점, 총점 1.6점)과 등급 판정 기준표와 [분석 결과 확정] 버튼을 출력한다.
8	사용자는 상세 점수 배정 현황이 등급 판정 기준표에 부합하는지 검토하고 [분석 결과 확정] 버튼을 클릭한다.
9	시스템은 "위험 분석 데이터가 성공적으로 반영되었습니다."라는 메시지를 출력한다. (UW-01:계약인수를 심사한다. Basic flow 6번으로 돌아간다.)
Alternative Flow	
Step	Interaction
A1	신규 가입자라 신용정보원에 조회 데이터가 없는 경우 1. 시스템은 "신용정보원에 해당 청약자의 조회 이력이 없습니다. 기본 위험등급(3등급)이 적용됩니다."라는 안내 메시지를 출력한다. 2. 사용자는 Basic Flow 5번으로 이동한다.
Exception Flow	
Step	Interaction
E1	신용정보원 시스템 연결이 실패한 경우 1. 시스템은 "신용정보원 시스템 연결에 실패하였습니다. 잠시 후 다시 시도해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 Basic Flow 2번을 재수행한다.

[CL-01:사고를 접수한다.]

Actors	보상처리팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 [보상관리] 탭을 클릭하여 하위 메뉴인 [신규 사고 접수 내역]을 선택한다.
2	시스템은 신규사고접수 입력란(기간, 상태)와 [조회] 버튼을 출력한다.
3	사용자는 신규사고접수 입력란에 기간(2026-04-19)과 상태(미처리)를 입력하고 [조회] 버튼을 누른다. [E1]
4	시스템은 미처리 사고 청구 목록 화면(접수 일시, 고객명, 청구 사유)과 [상세 조회] 버튼을 출력한다.
5	사용자는 검색창에 고객명(홍길동), 전화번호(010-1234-5678)를 입력하고 [상세 조회] 버튼을 누른다. [A1]
6	시스템은 사고 상세 정보 화면(제출된 사고 경위서, 증빙 서류 뷰어, 계약 원장 정보)과 [배당 담당자 검색] 버튼을 출력한다.
7	사용자는 사고 발생 지역 코드(SEOUL-01), 전문 분야(자동차 대물)를 입력하고 [배당 담당자 검색] 버튼을 누른다.
8	시스템은 현장조사역 후보 목록 화면(출동 가능 직원명, 미결 건수)과 [배당 및 접수 확정] 버튼을 출력한다.
9	사용자는 담당자 직원 번호(EMP-1023)를 입력하고 [배당 및 접수 확정] 버튼을 누른다.
10	시스템은 "담당자가 배당되었습니다." 안내 메시지를 출력한다.
Alternative Flow	
Step	Interaction
A1	필수 검색 조건이 선택되지 않은 경우 - 시스템은 "고객명과 전화번호는 필수 검색 조건입니다. 검색 조건을 추가해 주세요."라는 경고 메시지를 출력한다. - 사용자는 누락된 조건을 입력하고 Basic Flow 5번을 재수행한다.
Exception Flow	
Step	Interaction
E1	고객이 입력한 사고 정보 값의 형식이 허용 범위를 초과한 경우 - 시스템은 "입력된 사고 정보의 값이 허용 범위를 초과하였습니다."라는 경고 메시지를 출력하고 해당 입력란을 강조하여 출력한다. - 사용자는 정보 값을 수정하고 Basic Flow 3번을 재수행한다.

[CL-02:손해액을 산정한다.]

Actors	보상처리팀
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 【보상관리】 탭을 클릭하여 하위 메뉴인 【손해액 산정】을 선택한다.
2	시스템은 손해액 산정 입력란(담당자 사번, 접수번호)을 출력한다.
3	사용자는 손해액 산정 입력란에 담당자 사번(EMP-1023)과 접수 번호(ACC-2026-001)를 입력하고 【산정 대상 조회】 버튼을 누른다.
4	시스템은 손해액 산정 폼 화면(사고 초기 접수 내역)과 【피해 내역 조사 실행】 버튼을 출력한다.
5	사용자는 조사 진행 여부(Y)를 입력하고 【피해 내역 조사 실행】 버튼을 누른다. (<<include>> CL-03: 손해를 조사한다.) [A1]
6	시스템은 보상 한도액 화면(대인 한도 1,000만원, 대물 한도 2,000만원)과 최종 합의금, 자기부담금 입력란과 【최종 손해액 산출】 버튼을 출력한다. [E1]
7	사용자는 최종 합의금(1,500만원), 자기부담금(20만원)을 입력하고 【최종 손해액 산출】 버튼을 누른다.
8	시스템은 과실 상계 및 자기부담금이 공제된 최종 결정 손해액 화면(실 지급 보상금 1,480만원)과 【산정 내역 승인】 버튼을 출력한다.
9	사용자는 담당자 결제 의견(합의 완료 승인)을 입력하고 【산정 내역 승인】 버튼을 누른다.
10	시스템은 사정 내역 승인 완료 화면(지급 대기 상태값)과 【보험금 지급 실행】 버튼을 출력한다.
11	사용자는 【보험금 지급 실행】 버튼을 누른다. (<<extend>> CL-04: 보험금을 지급한다.)
12	시스템은 “보험금이 지급 완료되었습니다. 상태: 종결” 팝업창을 출력한다.
Alternative Flow	
Step	Interaction
A1	피해 조사 내역이 완료되지 않은 경우 1. 시스템은 "손해 조사가 완료되지 않아 산정을 진행할 수 없습니다. 조사를 완료해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 조사 완료 여부(Y)를 다시 체크하고 Basic Flow 3번을 재수행한다.
Exception Flow	
Step	Interaction
E1	입력된 합의금 한도 값이 허용 범위를 초과한 경우 1. 시스템은 "입력된 합의금 한도 값이 허용 범위를 초과하였습니다."라는 경고 메시지를 출력하고 해당 입력란을 강조하여 출력한다. 2. 사용자는 한도 값을 수정하고 Basic Flow 3번을 재수행한다.

[CL-03:손해를 조사한다.]

Actors	보상처리팀
Preconditions	CL-02: 손해액을 산정한다.
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 사고접수번호(ACC-2026-001)를 입력하고 [현장 조사 및 피해 입력] 버튼을 누른다. (CL-03: 손해액을 산정한다.에 의해 include)
2	시스템은 현장 조사 폼 화면(현장출동 소견, 파손 부위, 부상 정도)과 [보상 기준 확인] 버튼을 출력한다.
3	사용자는 현장조사소견(블랙박스 및 CCTV 분석결과 신호 대기 중 후미 추돌, 충격 부위 높이 및 파손 형상 일치함, 과거 파손 흔적(구상) 발견, 가해자 신호 위반으로 인한 일방 과실(100:0 추정), 파손 부위 코드(CAR-D-03), 부상 급수(12급)를 입력하고 [보상 기준 확인] 버튼을 누른다. [A1]
4	시스템은 보상 한도 범위 화면(예상 수리비:800,000원, 대인 보상 한도:1,000,000원)과 과실비율 입력 폼과 [과실 비율 검증] 버튼을 출력한다. [E1]
5	사용자는 당사 과실 비율(80%), 타사 과실 비율(20%)을 입력하고 [과실 비율 검증] 버튼을 누른다.
6	시스템은 과실 비율 총합 검증 결과 화면(100% 일치 확인), 면책여부 입력폼, [면/부책 판정] 버튼을 출력한다.
7	사용자는 면/부책 여부(부책)를 선택하여 입력하고 [면/부책 판정] 버튼을 누른다.
8	시스템은 손해 조사 내역 취합 화면(조사 보고서 초안)과 [조사 완료 및 저장] 버튼을 출력한다.
9	사용자는 최종 조사 의견(합의금 산출 진행 요망)을 입력하고 [조사 완료 및 저장] 버튼을 누른다.
10	시스템은 현장 조사 내역 저장 완료 팝업(조사 내역이 저장되었습니다. 일시:2026.04.23.10:31:22)을 출력한다. (CL-02: 손해액을 산정한다. Basic Flow 6번으로 이동한다.)
Alternative Flow	
Step	Interaction
A1	필수 피해 내역 코드가 선택되지 않은 경우 1. 시스템은 "파손 부위 코드는 현장 조사의 필수 입력란입니다. 코드를 리스트에 추가해주세요."라는 경고 메시지를 출력한다. 2. 사용자는 파손 부위 코드를 입력 리스트에 추가하고 Basic Flow 3번을 재수행한다.
Exception Flow	
Step	Interaction
E1	입력된 부상 급수 값이 약관 허용 범위를 초과한 경우 1. 시스템은 "입력된 급수 값이 허용 범위를 초과하였습니다."라는 경고 메시지를 출력하고 해당 입력란을 강조하여 출력한다. 2. 사용자는 급수 값을 수정하고 Basic Flow 3번을 재수행한다.

[CL-04:보험금을 지급한다.]

Actors	보상처리팀
Preconditions	CL-03: 손해액을 산정한다.
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 지급대기 중인 접수 목록 화면, 접수번호 입력란, [지급 정보 확인] 버튼을 출력한다.
2	사용자는 접수번호(ACC-2026-001)를 입력하고 화면 하단의 [지급 정보 확인] 버튼을 누른다.
3	시스템은 지급 대상 고객 정보 폼 화면(성명, 은행명, 계좌번호, 지급액)과 [계좌 유효성 검증] 버튼을 출력한다.
4	사용자는 수령 은행명(한국은행), 계좌번호(123-456-7890)를 입력하고 [계좌 유효성 검증] 버튼을 누른다. [A1]
5	시스템은 예금주 실명 일치 여부 결과 화면(검증 완료)과 [결재 상신] 버튼을 출력한다. [E1]
6	사용자는 결재용 사정의견서(의견서_ACC001.pdf)를 업로드하여 입력하고 [결재 상신] 버튼을 누른다.
7	시스템은 이체 품의서 요약 내역 화면(지급액, 수령 계좌정보)과 [최종 이체 승인] 버튼을 출력한다.
8	사용자는 결제 인증 비밀번호(1234)를 입력하고 [최종 이체 승인] 버튼을 누른다.
9	시스템은 은행 API 연동을 통한 실시간 계좌 이체 처리 경과 화면(Progress Bar)과 [사고 처리 종결] 버튼을 출력한다.
10	사용자는 이체 완료 여부(Y)를 확인하고 [사고 처리 종결] 버튼을 누른다.
11	시스템은 사고 건 지급 종결 내역 화면('지급 종결' 상태값, 알림톡 발송 완료 메시지)을 출력한다. (CL-02: 손해액을 산정한다.의 Basic Flow 12번으로 이동한다.)
Alternative Flow	
Step	Interaction
A1	필수 수령 계좌 정보가 입력되지 않은 경우 1. 시스템은 "수령 은행명과 계좌번호는 필수 사항입니다. 정보를 리스트에 추가해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 계좌 정보를 리스트에 추가하고 Basic Flow 4번을 재수행한다.
Exception Flow	
Step	Interaction
E1	입력된 계좌번호 자릿수 값이 허용 범위를 초과한 경우 1. 시스템은 "입력된 계좌번호 자릿수 값이 허용 범위를 초과하였습니다."라는 경고 메시지를 출력하고 해당 입력란을 강조하여 출력한다. 2. 사용자는 계좌번호 값을 수정하고 Basic Flow 3번을 재수행한다.

[CS-01:상품가입을 요청한다.]

Actors	고객
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 메뉴의 【상품조회】 탭을 클릭한다. (<<include>> CS-02: 보험상품을 조회한다.)
1	시스템은 본인확인 폼(이름, 주민등록번호, 전화번호)과 개인정보 처리 동의 항목(필수/선택)과 【다음】 버튼을 출력한다.
2	사용자는 본인 정보(이름: 박수현, 주민등록번호: 020101-3*****, 전화번호: 010-1234-5678)를 입력하고 동의 항목을 체크한 뒤 【다음】 버튼을 클릭한다
3	시스템은 보험에 가입할 차량번호 입력란과 【조회】 버튼을 출력한다.
4	사용자는 차량번호(64마0866)를 입력하고 【조회】 버튼을 누른다. [A1]
5	시스템은 조회된 차량의 정보(차량번호: 64마0866, 제조사: 현대, 모델명: 그랜저, 차종: 대형, 연료: 가솔린, 배기량: 2.999cc, 연식: 2020, 차량기준가액: 31,635,913원, 안전장치: 블랙박스, ABS)를 출력하고 【다음】 버튼을 출력한다.
6	사용자는 차량의 안전장치 여부를(블랙박스: O) 확인하고 【다음】 버튼을 누른다.
7	시스템은 차량의 운행 용도(출퇴근/가정용, 영업용, 업무용)와 운전자 범위 입력란(본인, 가족 한정)과 【예상보험료 계산】 버튼을 출력한다.
8	사용자는 운행용도를 출퇴근/가정용, 운전자 범위를 '본인 한정'으로 선택하고 【예상보험료 계산】 버튼을 클릭한다. [A2] (<<include>> CS-03: 예상 보험료를 계산한다.)
9	시스템은 "보험가입이 완료되었습니다." 메시지를 출력한다. [E2]
Alternative Flow	
Step	Interaction
A1	차량번호 조회 결과 차량 정보가 존재하지 않는 경우 1. 시스템은 "입력하신 차량번호로 차량 정보를 조회할 수 없습니다. 차량번호를 확인해주세요."라는 경고 메시지를 출력한다.
A2	운전자범위를 가족한정으로 선택한 경우 1. 시스템은 가족 정보 입력란(이름, 관계, 생년월일)을 출력한다. 2. 사용자는 이름(박수호), 관계(동생), 생년월일(070101)을 입력하고 【다음】 버튼을 누른다.
Exception Flow	
Step	Interaction
E1	가입 대상 연령 조건(만 20세 이상 39세 이하)에 해당하지 않는 경우 1. 시스템은 "해당 상품의 가입 대상 연령 조건에 해당하지 않아 가입이 제한됩니다."라는 경고 메시지를 출력한다.
E2	자동심사 결과 위험성이 높아 가입이 불가한 경우 1. 시스템은 "가입이 거절되었습니다. 자세한 사항은 고객 센터로 연락주세요.(1588-1000)"를 출력한다.

[CS-02:보험상품을 조회한다.]

Actors	고객
Preconditions	CS-01:상품가입을 요청한다.
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 판매 중인 자동차보험 상품 목록(MZ세대 다이렉트 개인용자동차보험,MZ세대 다이렉트 업무용자동차보험, MZ세대 다이렉트 영업용자동차보험)을 출력한다. (CS-01:상품가입을 요청한다.에 의해 include)
2	사용자는 목록에서 'MZ세대 다이렉트 개인용자동차보험'을 선택한다.
3	시스템은 상품 상세 화면(상품명:MZ 세대 다이렉트 개인용 자동차보험, 판매 기간: 2026-04-23 ~ 2026-05-01, 가입대상:만 20세 이상 39세 이하, 특약목록-마일리지 특약, 티맵안전운전 할인특약)과 [약관보기] 버튼, [가입하기] 버튼을 출력한다.
4	사용자는 특약 목록에서 조회하고 싶은 특약(마일리지 특약)을 클릭한다.
5	시스템은 특약의 상세내용(제목: 마일리지 특약 설명: 연간환산 운행거리 15,000 km이하 주행 시, 운행거리 실적에 따라 보험료를 환급 받을 수 있는 특약입니다.)을 출력한다.
6	사용자는 [약관보기] 버튼을 클릭한다.
7	시스템은 MZ 세대 다이렉트 개인용 자동차보험의 약관(보통약관, 특별약관) 설명서 PDF파일의 내용을 출력한다.
8	사용자는 PDF 문서를 확인하고 [가입하기] 버튼을 클릭한다. [A1]
9	시스템은 “보험 가입 화면으로 이동합니다” 팝업창을 출력한다. (CS-01:상품가입을 요청한다.의 Basic Flow 2번으로 이동한다.) [E1]
Alternative Flow	
Step	Interaction
A1	사용자가 약관 설명서를 저장할 경우 1. 사용자는 약관 문서에서 오른쪽 클릭을 하고 저장버튼을 누른다. 2. 시스템은 “PDF 파일이 저장되었습니다” 메시지를 출력한다.
Exception Flow	
Step	Interaction
E1	해당 상품이 판매 중지 상태인 경우 1. 시스템은 "해당 상품은 현재 판매가 중지된 상태입니다."라는 안내 메시지를 출력하고 Basic Flow 2번으로 돌아간다.

[CS-03:예상보험료를 산출한다.]

Actors	고객
Preconditions	CS-01:상품가입을 요청한다
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 상품에 적용할 할인 특약 목록(블랙박스 할인 특약, 커넥티드카 할인특약, 티맵안전운전 할인특약)과 [다음]을 출력한다. (CS-01:상품가입을 요청한다.에 의해 include)
2	사용자는 블랙박스 할인특약을 선택하고 [다음] 버튼을 누른다.
3	시스템은 피보험자/계약자 입력란(피보험자 정보, 계약자 정보)과 [다음] 버튼을 출력한다.
4	사용자는 피보험자 정보(이름: 박수현, 주민번호: 020101-*****, 연락처: 010-1234-5678)와 계약자 정보(박수현, 주민번호: 020101-*****, 연락처: 010-1234-5678)를 입력하고 [다음] 버튼을 누른다.
5	시스템은 할인 적용 항목(마일리지 가입, 3년 무사고 할인, ABS 특별요율)과 콤보박스, [다음] 버튼을 출력한다.
6	사용자는 할인 적용 항목(마일리지 가입-할인율:4.7%)을 선택하고 [다음] 버튼을 클릭한다. [A1]
7	시스템은 산출된 최종 보험료 확인 화면(대인배상1(의무): 312,892원, 대인배상: 560,114원, 대물배상: 938,805원, 자동차상해: 13,910원, 무보험차상해: 29,366원, 자기차량손해: 448,217원, 긴급출동서비스: 46,150원, 할인된 보험료: 499,883원, 최종 보험료: 1,849,571원)와 청약 내용 및 상품설명서와 [가입] 버튼을 출력한다. [E1]
8	사용자는 청약 내용 및 상품설명서를 확인하고 '이해하였음'에 체크한 뒤 [가입] 버튼을 클릭한다.
9	시스템은 "보험가입을 신청하는 중 입니다" 안내 메시지를 출력한다. (CS-01:상품가입을 요청한다.의 Basic Flow 9번으로 이동한다.)
Alternative Flow	
Step	Interaction
A1	고객이 할인 적용 요건을 충족하지 않을 경우 1. 시스템은 조회 결과, 할인 적용 기준(예: 티맵 점수 70점 이상)에 미달하여 할인을 적용할 수 없습니다."라는 안내 메시지를 출력한다. 2. 사용자는 해당 할인 항목의 체크를 해제하고 Basic Flow 6번으로 다시 수행한다.
Exception Flow	
Step	Interaction
E1	시스템 내부 오류로 최종 보험료 확인이 불가능한 경우 1. 시스템은 "지금은 보험료 계산을 할 수 없습니다. 나중에 다시 시도해주세요" 안내 메시지를 출력한다.

[CS-04:보험금을 청구한다.]

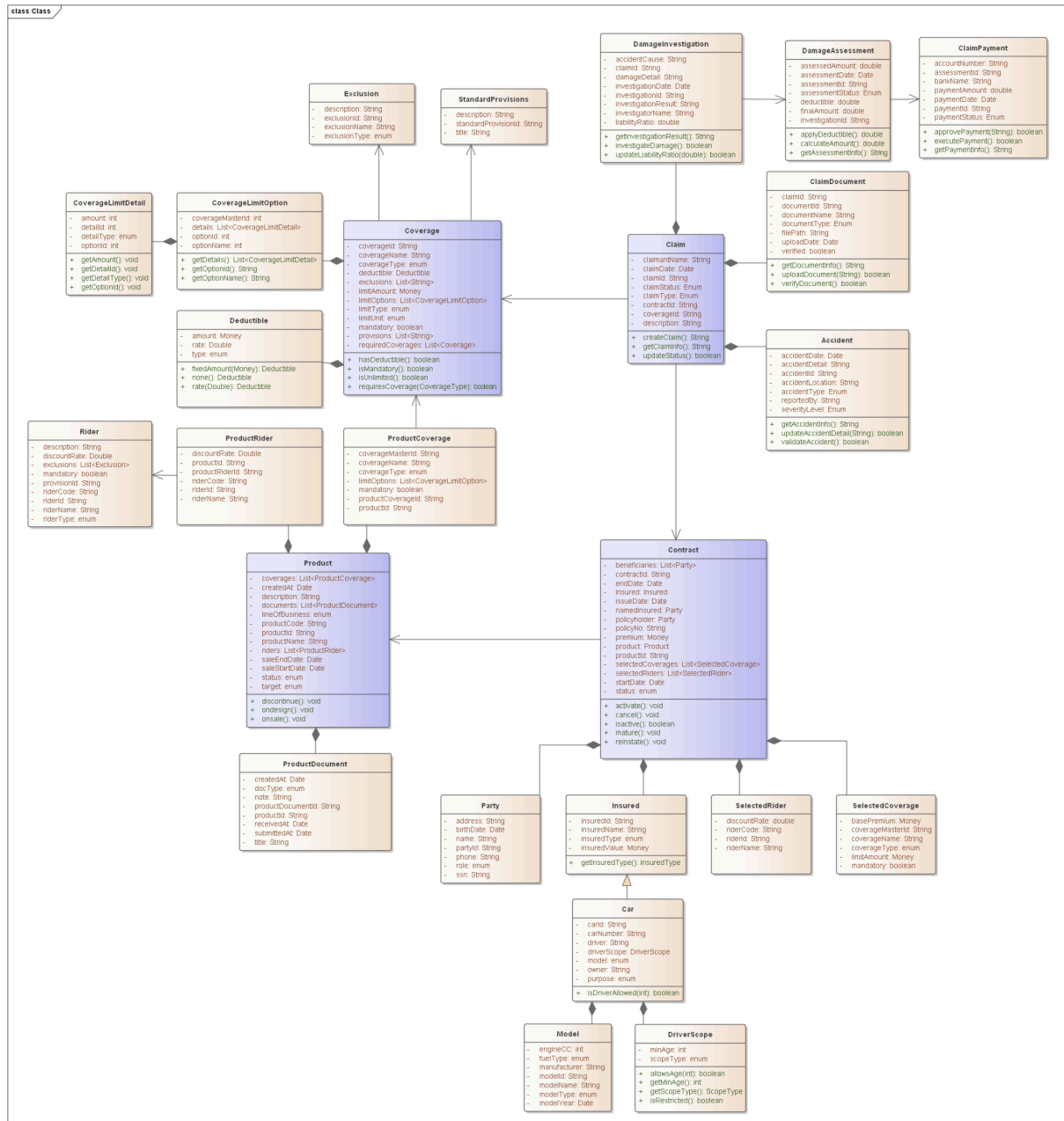
Actors	고객
Preconditions	
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	사용자는 메인 화면의 [사고접수 및 보험금 청구] 버튼을 누른다.
2	시스템은 본인 인증 수단 선택 화면(공동인증서, 간편비밀번호, 휴대폰 인증)과 [인증] 버튼을 출력한다.
3	사용자는 인증 수단을 선택하고 인증 정보(이름: 김민욱, 휴대전화번호: 010-1234-5678, 인증번호: 123456)를 입력하고 [인증] 버튼을 누른다.
4	시스템은 본인 인증 결과 화면(고객명 확인)과 [내 보험계약 확인] 버튼을 출력한다.
5	사용자는 계약 조회 동의(Y)를 입력하고 [내 보험계약 확인] 버튼을 누른다. (<<include>> CS-05: 보험계약을 조회한다.) [A1]
6	시스템은 사고 기초 정보 입력 폼 화면(사고 일시, 사고 장소, 상세 경위)과 [다음] 버튼을 출력한다.
7	사용자는 사고 발생 일시(2026-04-19 14:00), 사고 장소(서울시 강남구), 상세 경위(후방 추돌)를 입력하고 [다음] 버튼을 누른다.[E1]
8	시스템은 필수 항목 누락 검증 결과 화면(검증 통과)과 증빙 서류 업로드 폼 화면(진단서, 현장사진) 및 [서류 제출] 버튼을 출력한다.
9	사용자는 진단서 파일(진단서_김고객.jpg), 현장사진 파일(현장_1.jpg)을 첨부하여 입력하고 [서류 제출] 버튼을 누른다.
10	시스템은 첨부 파일 썸네일 뷰어 화면(정상 첨부 확인)과 최종 제출 동의 선택란과 [최종 청구 완료] 버튼을 출력한다.
11	사용자는 최종 제출 동의(Y)를 체크하고 [최종 청구 완료] 버튼을 누른다.
12	시스템은 보험금 청구 완료 화면(부여된 접수 번호, 보상팀 이관 안내)과 "보험금 청구가 완료되었습니다" 상태 메시지를 띄운다.
Alternative Flow	
Step	Interaction
A1	청구 대상 보험 계약이 선택되지 않은 경우 1. 시스템은 "대상 보험 계약은 필수 선택 사항입니다. 대상을 리스트에 추가해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 대상 계약을 체크하여 추가하고 Basic Flow 5번을 재수행한다.
Exception Flow	
Step	Interaction
E1	입력된 사고 발생 일시 값이 보험 가입 기간의 허용 범위를 초과한 경우(가입 전 사고) 1. 시스템은 "입력된 사고 일시 값이 허용 범위를 초과하였습니다."라는 경고 메시지를 출력하고 해당 입력란을 강조하여 출력한다. 2. 사용자는 일시 값을 수정하고 Basic Flow 7번을 재수행한다.

[CS-05:보험계약을 조회한다.]

Actors	고객
Preconditions	CS-04:보험금을 청구한다.
Postconditions	
Trigger	
Basic Flow	
Step	Interaction
1	시스템은 본인 인증 수단 폼 화면(공동인증서, 간편비밀번호, 휴대폰 인증)과 [인증] 버튼을 출력한다. (CS-04: 보험금을 청구한다.에 의해 include)
2	사용자는 인증수단을 휴대폰 인증을 선택하고 이름(박수현), 주민등록번호 (020101-3*****), 휴대폰번호(010-1234-5678), 인증번호(123456)을 입력하고 [인증] 버튼을 누른다. [A1]
3	시스템은 “인증이 완료되었습니다” 메시지와 함께 보험계약 조회 화면(조회 기간, 계약 상태)와 [조회] 버튼을 출력한다.
4	사용자는 조회조건(기간: 전체, 계약 상태: 유지 중)을 입력하고 [조회] 버튼을 누른다.
5	시스템은 조건에 맞는 보험 계약 목록 화면(증권 번호:IN-2026-001, 상품명: MZ세대 개인용 자동차 보험)과 [상세 내역 확인] 버튼을 출력한다. [E1]
6	사용자는 증권 번호가(IN-2026-001)인 계약을 선택하고 [상세 내역 확인] 버튼을 누른다.
7	시스템은 계약의 상세내역(계약일시(2026-04-01), 보험료(2,509,200원), 담보내용(대인배상1, 대인배상2, 대물배상), 특약목록(마일리지 특약))과 [청구] 버튼을 출력한다.
8	사용자는 상세내역을 확인하고 [청구] 버튼을 누른다.
9	시스템은 "청구 프로세스로 이동합니다." 메시지를 출력한다. (CS-04: 보험금을 청구하다. 의 Basic Flow 6번으로 이동한다.)
Alternative Flow	
Step	Interaction
A1	인증 수단이 선택되지 않은 경우 1. 시스템은 "본인 인증 수단은 필수 사항입니다. 인증 수단을 리스트에 추가해 주세요."라는 경고 메시지를 출력한다. 2. 사용자는 인증 수단을 선택하여 추가하고 Basic Flow 2번을 재수행한다.
Exception Flow	
Step	Interaction
E1	시스템 내부 오류로 계약 상세 확인이 불가능한 경우 1. 시스템은 “시스템 내부 오류로 계약 상세 확인이 불가능합니다”라는 안내메시지를 출력한다.

3 설계 모델링

3.1 Class Diagram



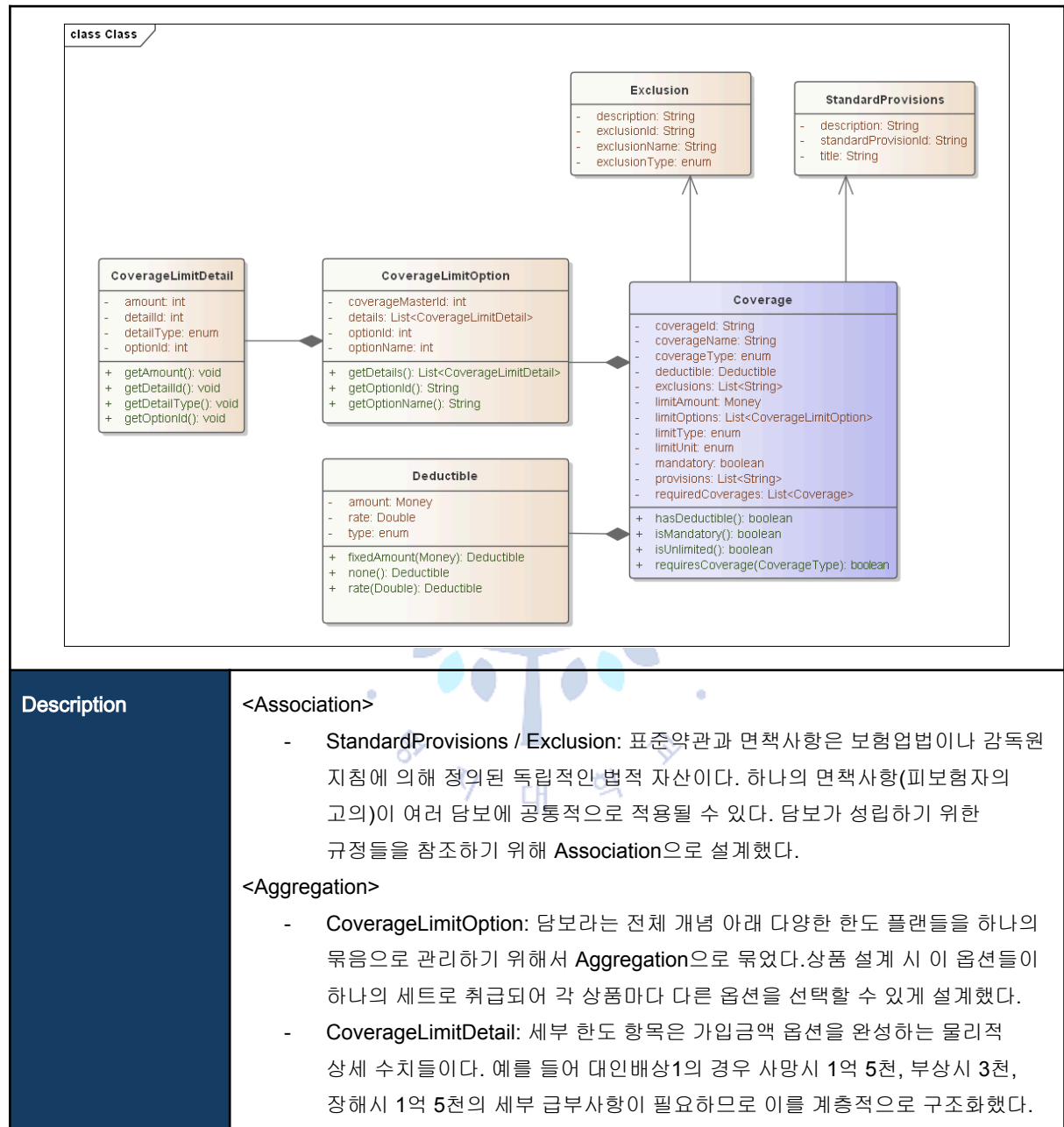
Association: 객체가 지속적으로 영향을 미치는 관계를 표현한다.

Generalization: 공통 속성과 역할을 상위 클래스로 추출하여 하위 클래스가 이를 상속받는 관계를 표현한다.

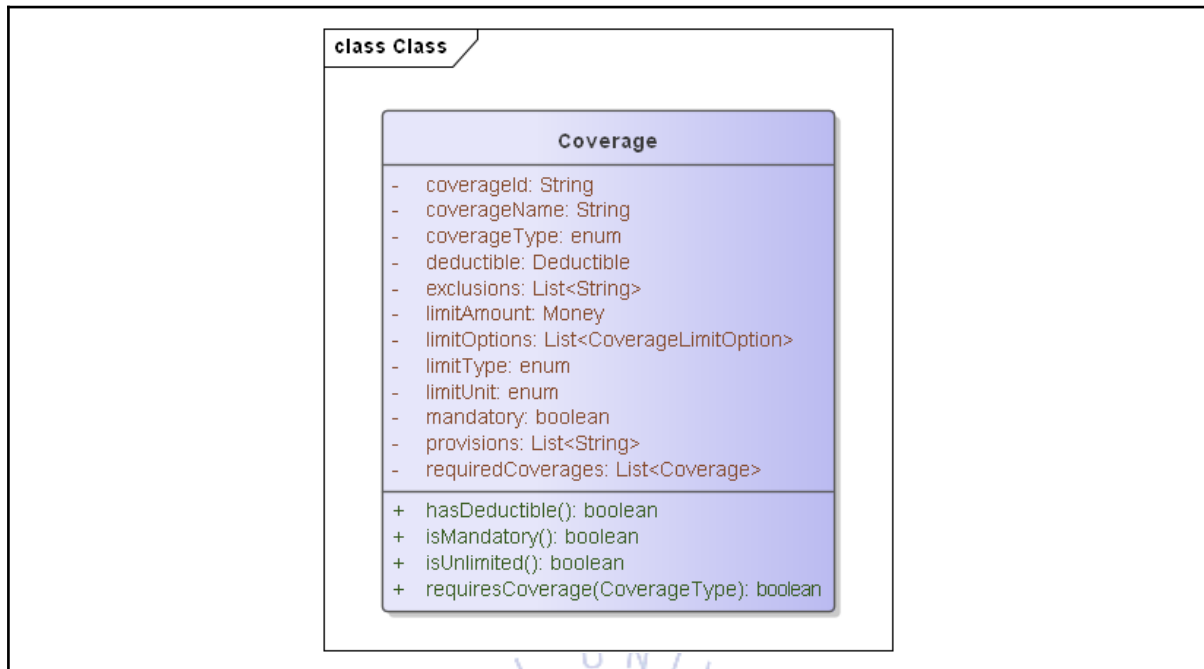
Aggregation: 전체-부분 관계를 표현하며 부분이 전체의 생명주기를 함께한다. 다만 본 설계에서는 부분이 전체 외부에서도 독립적으로 존재할 수 있는 경우에도 **Aggregation**을 적용하여 개념적인 소속 관계를 표현했다.

3.2 Class 관계 및 설명

Coverage 관계



Coverage

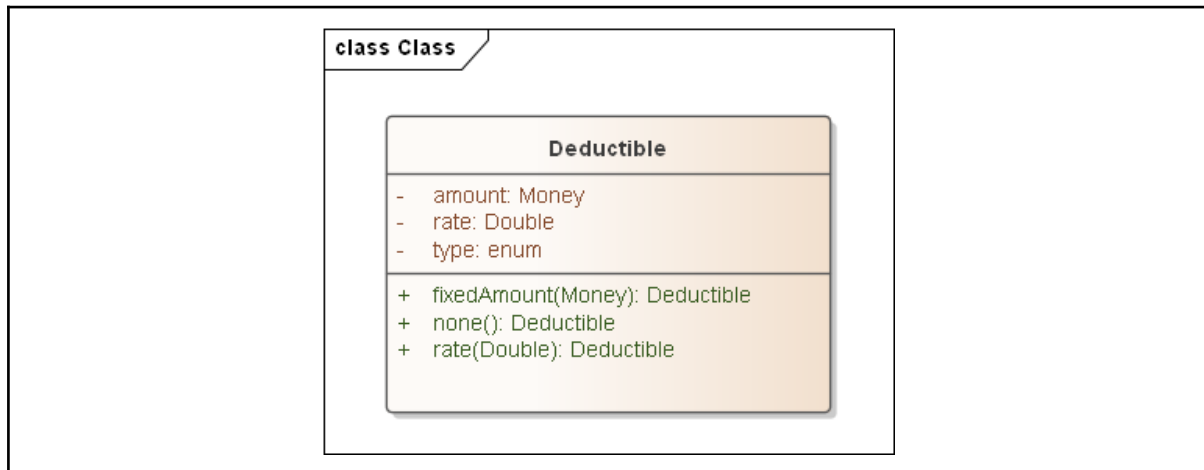


Description	담보는 보험계약에서 보험사가 보상을 약속하는 위험의 단위다. 하나의 보험상품은 여러 담보로 구성되며 각 담보는 독립적인 보상 조건과 한도를 가진다. 담보는 보험료 산출의 기본 단위이자 계약자가 선택하고 조합할 수 있는 보장의 최소 구성 요소이기 때문에 클래스로 추출했다.	
Attributes	Coverage의 속성은 크게 세 그룹으로 나뉜다. 담보 식별 정보 (coverageId, coverageName, coverageType), 보상 조건 (limitAmount, limitType, limitUnit, deductible), 담보 간 제약 관계 (mandatory, requiredCoverages, exclusions, provisions)로 구성된다. 특히 requiredCoverages는 예를 들어 대인배상Ⅱ가 대인배상Ⅰ 가입을 전제로 하는 것처럼 담보 간 의존 규칙을 도메인 수준에서 표현하기 위해 추가했다.	
Field	Type	Description
coverageId	String	담보 ID
coverageName	String	담보 이름
coverageType	enum	담보 종목(대인배상Ⅰ, 대인배상Ⅱ, 대물배상, 자동차상해, 자기차량손해, 무보험차상해)
deductible	Deductible	자기부담금
exclusions	List<String>	면책사항 ID의 모음
limitAmount	Money	한도액
limitOptions	List	담보 옵션 모음

limitType	enum	한도 유형(무한/고정/차량가액)
limitUnit	enum	한도 단위(인당/사고당/복합)
mandatory	boolean	필수 여부
provisions	List<String>	참조 약관 ID의 모음
requiredCoverages	List<Coverage>	필수로 선택되어야 하는 담보

Operation	외부에서 조건 분기를 처리하지 않도록 담보 스스로 자신의 상태를 판단하게 설계했다. isMandatory(), hasDeductible(), isUnlimited(), requiresCoverage() 모두 단순 getter가 아니라 비즈니스 규칙을 캡슐화한 판단 오퍼레이션이다.	
Return type	Method	Description
boolean	hasDeductible	자기부담금이 있는지 확인한다.
boolean	isUnlimited	한도가 없는지 확인한다.
boolean	isMandatory	필수로 들어야하는 담보인지 확인한다.
boolean	requiresCoverage	필수 담보가 있는지 확인한다.

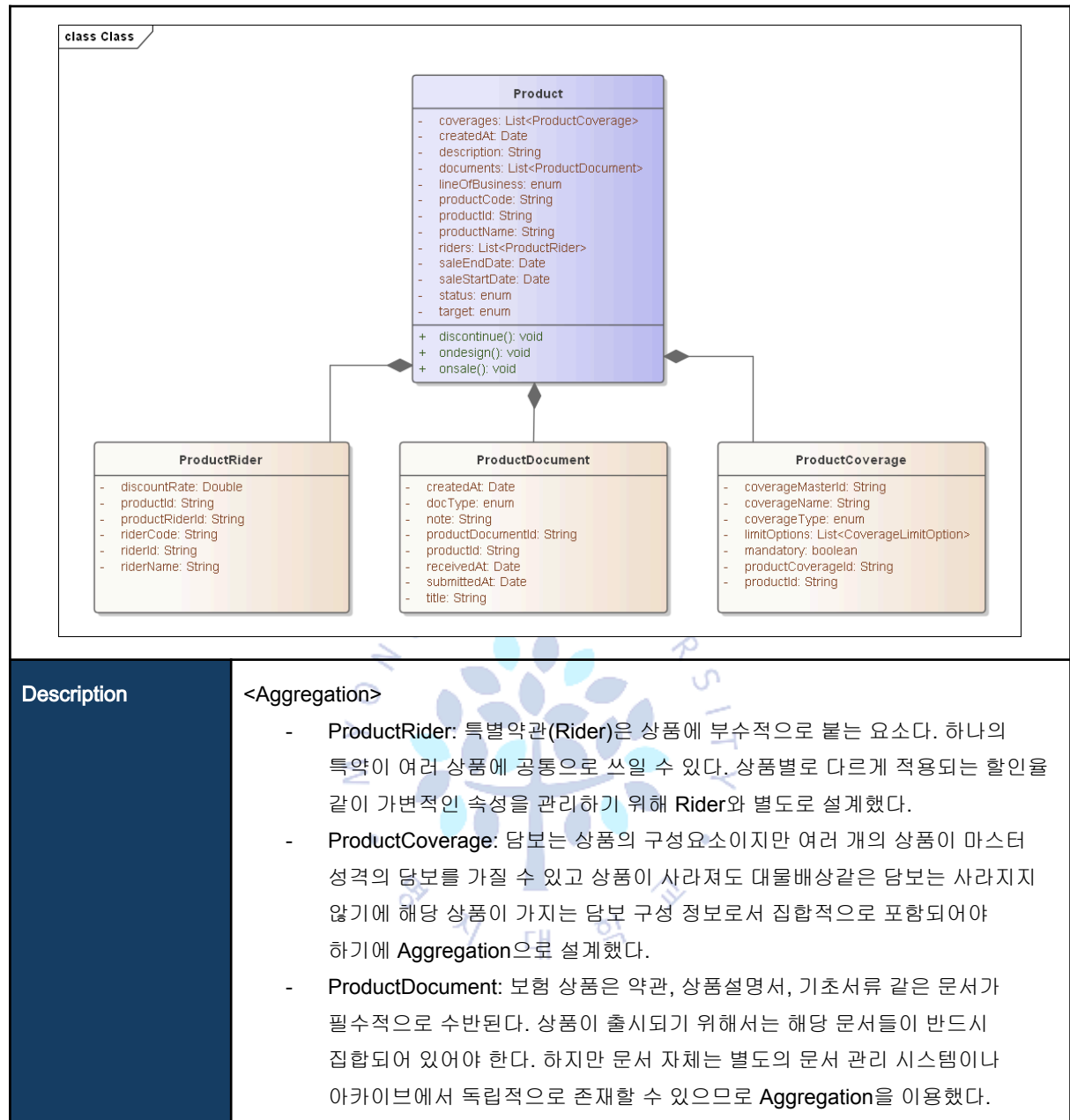
Deductible



Description	자기부담금은 정액/비율/없음처럼 계산 방식의 차이를 내포하는 독립적인 도메인 개념이다. 이 계산 방식의 차이를 Coverage 안에 넣으면 조건 분기가 Coverage 에 생기므로 별도 클래스로 분리했다.	
Attributes	자기부담금의 속성은 두 가지 계산 방식을 모두 표현하기 위해 구성했다. amount 는 정액 방식일 때의 고정 금액, rate 는 비율 방식일 때의 적용 비율이다. type 은 현재 어떤 방식이 적용되는지를 구분하는 식별자다.	
Field	Type	Description
amount	Money	자기부담금 액수
rate	Double	자기부담금 비율
type	enum	자기부담금 유형(없음/정액/비율)

Operation	자기부담금의 메서드는 자기부담금 객체를 생성하는 팩토리 방식으로 설계했다. fixedAmount(Money) 는 정액형, rate(Double) 는 비율형, none() 은 자기부담금 없음을 각각 명시적으로 생성한다. 이렇게 하면 호출하는 쪽에서 type 을 직접 세팅하는 대신 의도가 명확한 메서드를 통해 객체를 얻게 되어 잘못된 상태의 Deductible 이 만들어지는 것을 방지할 수 있다.	
Return type	Method	Description
Deductible	none	자기부담금 없을 때 쓰는 static 메서드
Deductible	rate	자기부담금 유형이 비율로 정해질 때 쓰는 static 메서드
Deductible	fixedAmount	자기부담금이 고정금액일 때 쓰는 static 메서드

Product 관계



Product

class Class Product - coverages: List<ProductCoverage> - createdAt: Date - description: String - documents: List<ProductDocument> - lineOfBusiness: enum - productCode: String - productId: String - productName: String - riders: List<ProductRider> - saleEndDate: Date - saleStartDate: Date - status: enum - target: enum + discontinue(): void + ondesign(): void + onsale(): void		
Description	보험상품은 보험사가 판매하는 보장 패키지의 단위로 담보·특약·문서를 하나의 틀로 묶어 감독원에 신고하고 시장에 출시하는 독립적인 도메인 개념이다. 단순한 데이터 묶음이 아니라 설계 → 판매 → 판매중단 이라는 명확한 생명주기를 가지며 그 상태 전이를 스스로 관리하는 행위 주체이므로 독립 클래스로 추출했다.	
Attributes	속성은 세 그룹으로 나뉜다. 상품 식별 정보 (productId, productCode, productName, lineOfBusiness)는 상품을 다른 상품과 구별하기 위해 필요하다. 판매 조건 (saleStartDate, saleEndDate, status, target)은 언제, 누구에게 팔 수 있는지를 표현하며 판매 유효성 판단의 근거가 된다. 구성 요소 (coverages, riders, documents)는 앞서 설명한 Aggregation 관계로 상품이 출시되기 위해 반드시 갖춰야 할 요소들이다. description과 createdAt은 상품 관리와 이력 추적을 위한 부가 정보다.	
Field	Type	Description
productId	String	상품 ID
productName	String	상품 이름(MZ세대 다이렉트 개인용자동차보험)
status	enum	상품 상태(설계중/요율검증중/인가신청/인가완료/판매확정/판매중)
lineOfbussiness	enum	보험 종류(개인용자동차보험/영업용자동차보험/업무용자동차보험/이륜차)
coverages	List	상품이 갖는 담보들(자동차상해, 대인배상1,2, 대물배상, 무보험차상해)

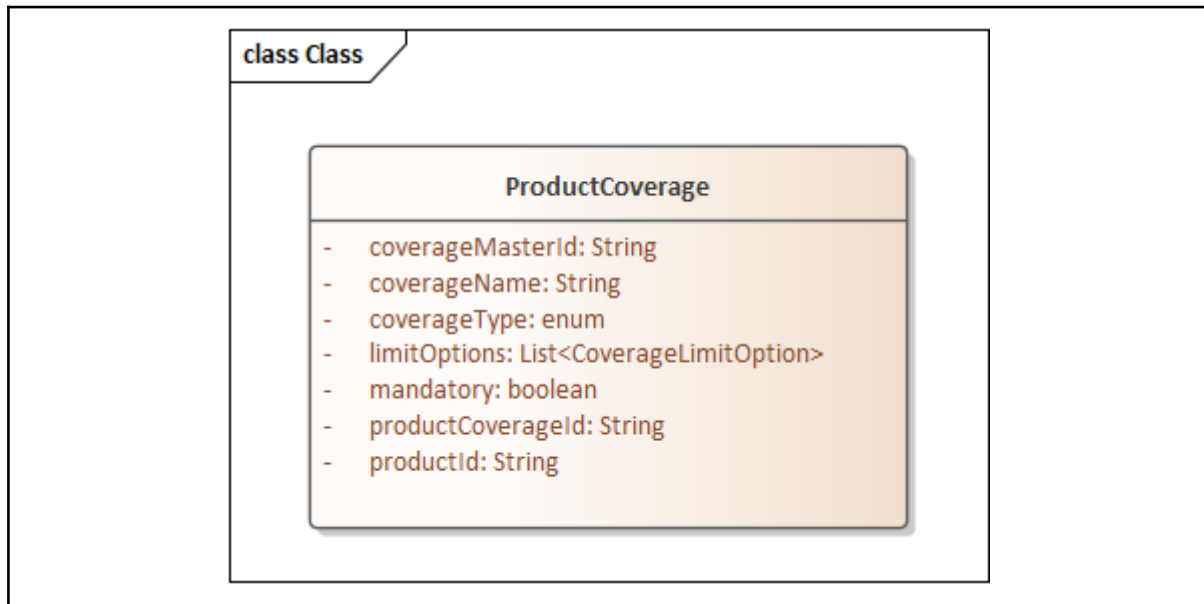
createdAt	Date	상품 생성 날짜
description	String	상품 설명
documents	List	상품에 관한 문서들(상품 설명서, 보험료 및 책임준비금 산출서)
productCode	String	상품 코드
riders	List	특약 목록(블랙박스 장착 특약, 마일리지 특약)
saleEndDate	Date	상품 판매 종료일
saleStartDate	Date	상품 판매 개시일
target	enum	가입대상(만 22세 이상/만 26세 이상)

Operation	상품의 메서드는 상품의 생명주기 상태 전이를 표현하도록 설계했다. ondesign()은 상품이 설계 중 상태로 진입함을, onsale()은 감독원 신고 후 판매 가능 상태로 전환됨을, discontinue()는 판매 중단 처리를 의미한다. 이 세 메서드가 status 변경의 유일한 진입점이 되어 외부에서 status를 직접 조작하는 것을 막고 상태 전이 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
void	ondesign	상품 상태를 설계 중으로 전환하는 메서드
void	onsale	상품 상태를 판매 중으로 전환하는 메서드
void	discontinue	상품 상태를 판매 중단으로 전환하는 메서드

ProductRider

class Class ProductRider - discountRate: Double - productId: String - productRiderId: String - riderCode: String - riderId: String - riderName: String		
Description	특약은 여러 상품에 공통으로 존재하는 마스터 개념이지만 상품마다 적용되는 할인율이 다르다. 이 상품별 가변 속성을 Rider 마스터에 넣으면 마스터가 오염되고 Product에 직접 넣으면 상품과 특약의 관계를 표현할 수 없다. 따라서 Product와 Rider 사이의 관계 자체를 하나의 클래스로 추출했다.	
Attributes	riderId와 riderCode riderName은 Rider 마스터를 참조하기 위한 식별 정보다. productId는 어느 상품에 속한 관계인지 명시한다. discountRate는 이 관계에서만 의미를 가지는 상품별 가변 속성이다.	
Field	Type	Description
discountRate	Double	할인율
productId	String	상품 ID
productRiderId	String	상품에 적용된 특약 ID
riderCode	String	특약의 코드
riderId	String	특약 ID
riderName	String	특약의 이름

ProductCoverage

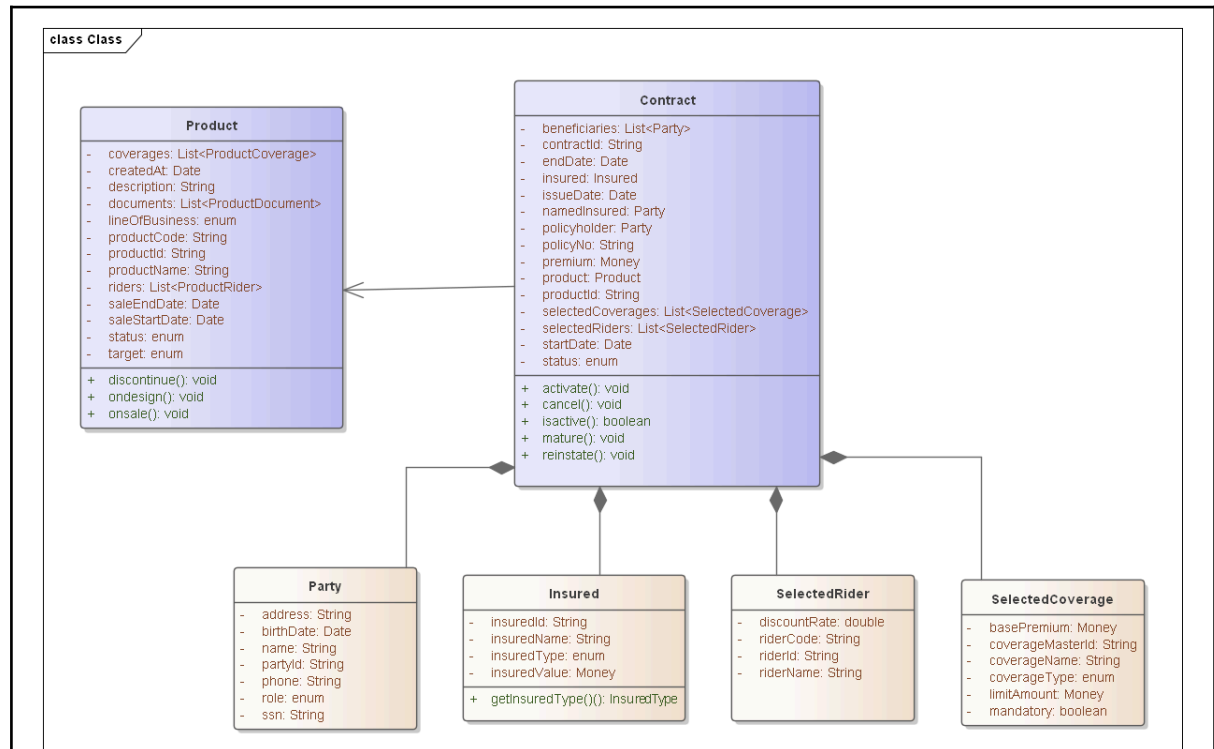


Description	ProductRider와 같은 맥락으로 담보 마스터는 여러 상품에 공통으로 존재하는 독립적인 도메인 개념이다. 그러나 의무담보 여부나 선택 가능한 한도 옵션은 상품마다 다르게 구성된다. 이 상품별 맥락 정보를 Coverage 마스터에 넣으면 마스터가 오염되므로 Product와 Coverage 사이의 관계를 별도 클래스로 추출했다.	
Attributes	coverageMasterId와 coverageName coverageType은 Coverage 마스터를 참조하기 위한 식별 정보다. productId와 productCoverageId는 어느 상품에 속한 관계인지를 명시한다. mandatory는 해당 상품에서 이 담보가 의무인지 선택인지를 나타내는 상품별 가변 속성이고 limitOptions는 동일한 담보라도 상품마다 제공하는 한도 옵션이 다를 수 있기 때문에 여기서 관리하도록 설계했다.	
Field	Type	Description
productCoverageId	String	상품에 적용된 담보 ID
productId	String	상품 ID
coverageMasterId	String	마스터 담보의 ID
coverageName	String	담보의 이름
coverageType	enum	담보의 종목(대인배상I, 대인배상II, 대물보상, 자동차상해, 무보험차상해)
limitOptions	List	가입금액 옵션(기본형/고급형)
mandatory	boolean	담보의 필수 여부

ProductDocument

class Class ProductDocument - createdAt: Date - docType: enum - note: String - productDocumentId: String - productId: String - receivedAt: Date - submittedAt: Date - title: String		
Description	보험상품은 보험 약관, 상품설명서, 기초서류, 요율확인서 같은 문서를 감독원에 제출하고 승인받아야 출시할 수 있다. 이 문서들은 단순한 첨부파일이 아니라 제출일, 접수일, 문서 유형이라는 독자적인 속성과 이력을 가지는 도메인 개념이므로 독립 클래스로 추출했다.	
Attributes	속성은 두 그룹으로 나뉜다. 문서 식별 및 분류 정보(productDocumentId title docType)는 어떤 종류의 문서인지를 구분하기 위해 필요하다. 이력 관리 정보(submittedAt receivedAt createdAt)는 감독원 제출과 접수 시점을 추적하며 기초서류 제출 의무를 이행했음을 증빙하는 근거가 된다. note는 보완 요청 사항을 기록하기 위한 부가 정보이고 productId는 어느 상품에 속한 문서인지를 명시한다.	
Field	Type	Description
productDocumentId	String	상품의 문서 ID
productId	String	상품 ID
docType	enum	문서유형(사업방법서, 보험약관, 보험설명서, 요율확인서, 인가신청서)
title	String	제목
note	String	비고
createdAt	Date	생성날짜
receivedAt	Date	수령일
submittedAt	Date	제출일

Contract 관계



Description

연관관계 설명

<Association>

- **Product**: 상품은 계약과 독립적인 생명주기를 가지므로 **Association**으로 설계했다. 계약 시점의 담보 및 특약 구성은 **SelectedCoverage**, **SelectedRider**로 별도 스냅샷을 두어 이후 상품 개정의 영향을 받지 않도록 분리했다.

<Aggregation>

- **Party**: 계약자, 피보험자, 보험수익자, 보험자를 **Party**로 일반화하고 **role**로 구분했다. 계약당사자(**Party**)는 계약 외부에서도 존재할 수 있으므로 **Aggregation**으로 설계했다.
- **Insured**: 피보험체를 **Insured**로 일반화하여 차량, 이륜차, 농기계처럼 다양한 보험 대상을 상속으로 확장할 수 있게 했다. 피보험체 역시 계약에 종속되지 않으므로 **Aggregation**으로 설계했다.
- **SelectedCoverage / SelectedRider**: 계약 체결 시점에 확정된 담보와 특약 구성을 계약 안에서 함께 관리하기 위해 **Aggregation**으로 설계했다.

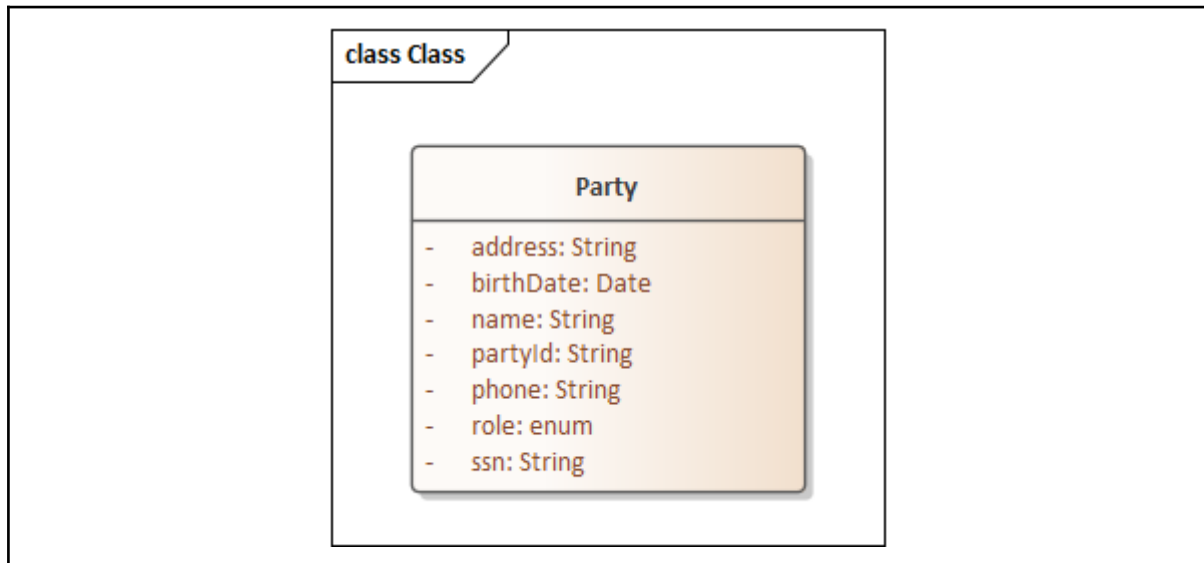
Contract

class Class Contract - beneficiaries: List<Party> - contractId: String - endDate: Date - insured: Insured - issueDate: Date - namedInsured: Party - policyholder: Party - policyNo: String - premium: Money - product: Product - productId: String - selectedCoverages: List<SelectedCoverage> - selectedRiders: List<SelectedRider> - startDate: Date - status: enum + activate(): void + cancel(): void + isactive(): boolean + mature(): void + reinstate(): void		
Description	보험계약은 계약자와 보험사 사이의 법적 효력을 가지는 보장 약속으로 상품·당사자·담보·특약을 하나로 묶어 법적 구속력을 부여하는 독립적인 도메인 개념이다. 단순한 데이터 묶음이 아니라 청약 → 활성 → 만기/해지/부활 이라는 명확한 생명주기를 갖고 상태를 스스로 관리하는 행위 주체이므로 독립 클래스로 추출했다.	
Attributes	Contract의 속성은 세 그룹으로 나뉜다. 계약 식별 및 기간 정보(contractId, policyNo, issueDate, startDate, endDate, status)는 계약의 동일성과 유효 기간을 표현한다. 계약 당사자(policyholder, namedInsured, insured, beneficiaries)는 계약에서 각기 다른 법적 역할을 수행하는 주체다. 계약 내용(product, productId, selectedCoverages, selectedRiders, premium)은 계약자가 선택한 보장 구성과 보험료를 담는다. selectedCoverages와 selectedRiders는 계약 시점의 담보, 특약 구성을 스냅샷으로 고정하여 이후 상품이 변경되어도 영향을 받지 않도록 했다.	
Field	Type	Description
contractId	String	계약 ID
issuedDate	Date	증권 발행일
startDate	Date	보험 개시일
endDate	Date	계약 만기일

policyholder	Party	계약자
namedinsured	Party	기명피보험자
beneficiaries	List	보험수익자
insured	Insured	보험대상물
productId	String	보험 상품 ID
selectedCoverages	List	계약 시 선택된 담보들
selectedRiders	List	계약 시 선택된 특약들
premium	Money	보험료
policyNo	String	증권번호
status	enum	계약상태(청약/미납/해지/활성/만기/부활)

Operation	activate(), cancel(), mature(), reinstate()는 계약의 생명주기 상태 전이를 표현한다. 이 메서드들은 외부에서 status를 직접 조작하는 것을 막고 상태 전이 규칙을 클래스 안에 캡슐화한다. isActive()는 계약이 현재 유효한 상태인지를 계약 스스로 판단하게 한다.	
Return type	Method	Description
void	activate	초회보험료 납입 확인 시 계약 유지 상태로 전환하는 메서드
void	cancel	계약을 해지상태로 전환하는 메서드
void	mature	계약을 만기상태로 전환하는 메서드
void	reinstate	계약을 부활상태로 전환하는 메서드
boolean	isActive	유지 중인 계약인지 확인하는 메서드

Party



Description	Party는 계약 당사자를 의미한다. 계약자, 피보험자, 보험수익자는 계약에서 각기 다른 법적 역할을 수행하지만 인적 사항이라는 공통 속성을 가진다. 이를 Party로 일반화하여 동일 인물이 여러 역할을 수행하는 경우를 자연스럽게 표현할 수 있도록 독립 클래스로 추출했다.	
Attributes	Party의 속성은 두 그룹으로 나뉜다. 식별 및 개인 정보(partyId, name, birthDate, ssn, address, phone)는 당사자를 식별하고 본인 확인에 필요한 인적 사항을 저장하고 ssn은 계약 심사 시 외부 기관과의 본인 조회에 활용된다. 역할 정보(role)는 해당 당사자가 계약 내에서 어떤 역할을 맡는지 구분한다. 이를 통해 하나의 Party가 계약에 따라 다른 역할로 참여할 수 있도록 한다.	
Field	Type	Description
partyId	String	당사자 ID
address	String	주소
birthDate	Date	생일
name	String	이름
phone	String	휴대전화번호
role	enum	역할(피보험자, 계약자, 보험수익자)
ssn	String	주민등록번호

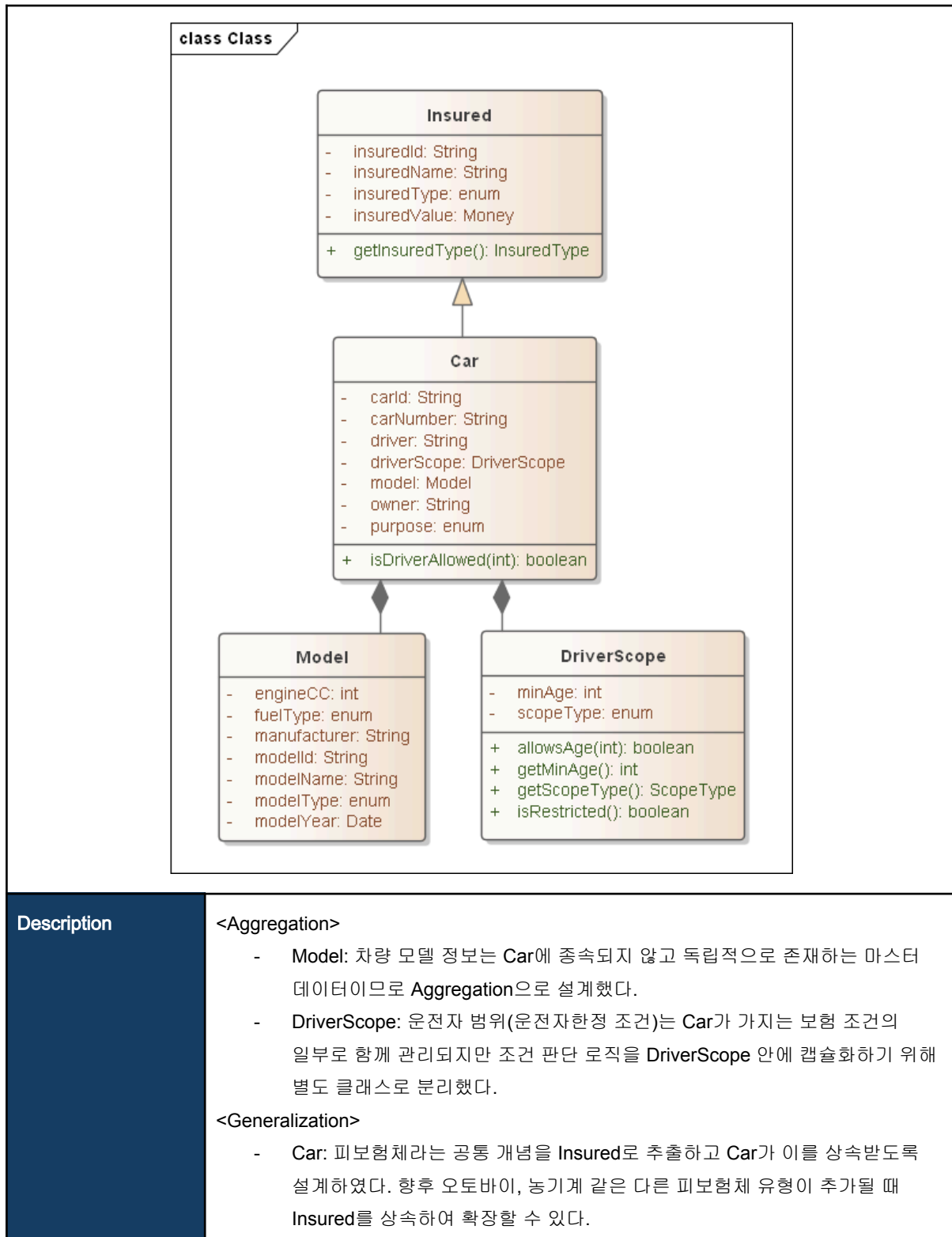
SelectedRider

class Class SelectedRider - discountRate: double - riderCode: String - riderId: String - riderName: String		
Description	계약 체결 시점에 확정된 특약 구성을 스냅샷으로 보존하기 위해 독립 클래스로 추출했다. Rider 마스터가 이후 변경되더라도 계약 당시의 특약 조건이 유지되어야 하기 때문이다.	
Attributes	riderId riderCode riderName은 어떤 특약인지를 식별하기 위한 정보고 discountRate는 계약 시점에 확정된 할인율을 저장하여 이후 상품 변경과 무관하게 계약 조건을 유지한다.	
Field	Type	Description
riderId	String	특약 ID
riderName	String	특약 이름
discountRate	double	할인율
riderCode	String	특약 코드

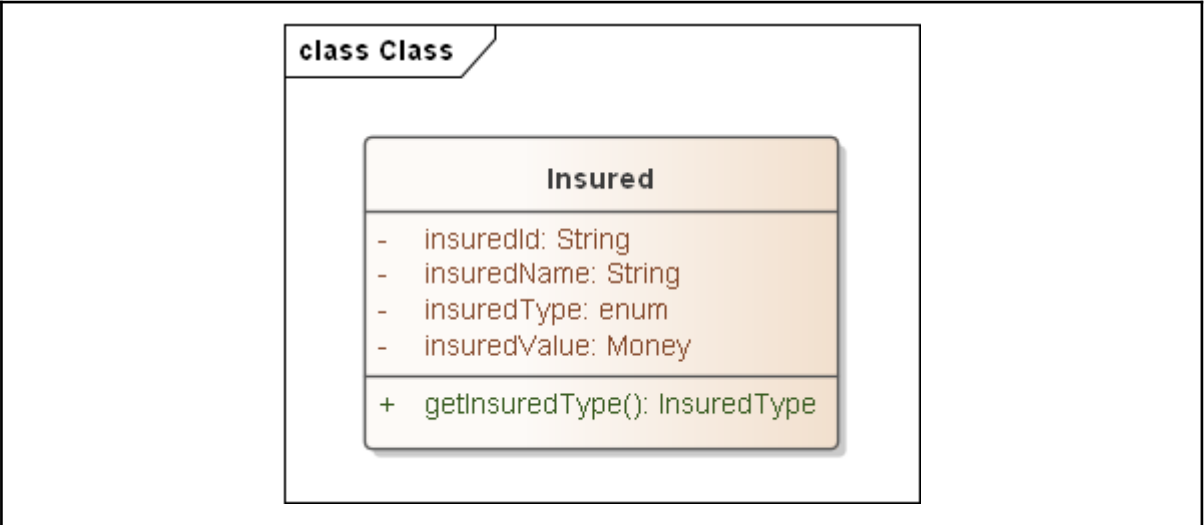
SelectedCoverage

class Class SelectedCoverage - basePremium: Money - coverageMasterId: String - coverageName: String - coverageType: enum - limitAmount: Money - mandatory: boolean		
Description	계약 체결 시점에 확정된 담보 구성을 스냅샷으로 보존하기 위해 독립 클래스로 추출했다. Coverage 마스터가 이후 변경되더라도 계약 당시의 담보 조건이 유지되어야 하기 때문이다.	
Attributes	속성은 두 그룹으로 나뉜다. 식별 및 분류 정보(coverageMasterId coverageName coverageType mandatory)는 어떤 담보인지와 필수/선택 여부를 구분하며 coverageMasterId를 통해 원본 담보 정보와 연결된다. 금액 정보(basePremium limitAmount)는 계약 시점에 확정된 보험료와 보상 한도를 저장하여 이후 약관 변경과 무관하게 계약 조건을 유지한다.	
Field	Type	Description
basePremium	Money	기본 보험료
coverageMasterId	String	담보의 마스터 ID
coverageName	String	담보 이름
coverageType	enum	담보의 종목(담보 종목 대인배상I, 대인배상II, 대물배상, 자동차상해, 자기차량손해, 무보험차상해)
limitAmount	Money	한도액
mandatory	boolean	담보의 필수 여부

Insured 관계



Insured



Description	Insured는 보험이 보장하는 대상물이다. 계약과 독립적인 생명주기를 가지는 도메인 개념이므로 독립 클래스로 추출했다. Car는 자동차보험의 구체적인 피보험체다. 향후 오토바이같은 다른 피보험체 유형을 상속으로 확장하기 위해 Insured를 상위 클래스로 두었다.	
Attributes	Insured의 속성은 두 그룹으로 나뉜다. insuredId와 insuredName은 피보험물을 식별하기 위한 정보이고, insuredType과 insuredValue는 피보험물의 종류를 구분하고 계약 시점의 평가 가액을 저장하여 보상 한도 산정의 기준이 된다.	
Field	Type	Description
insuredId	String	보험대상물 ID
insuredName	String	보험대상물 이름
insuredType	enum	보험대상물 종류(차량, 오토바이)
insuredValue	Money	보험대상물의 가치(차량의 경우 차량가액)

Operation	getInsuredType()은 계약 심사 및 보상 처리 단계에서 피보험물 종류에 따른 분기 처리가 가능하도록 insuredType 값을 외부에 제공한다.	
Return type	Method	Description
InsuredType	getInsuredType	보험대상물의 종류를 반환하는 메서드

Car

class Class Car - carId: String - carNumber: String - driver: String - driverScope: DriverScope - model: Model - owner: String - purpose: enum + isDriverAllowed(int): boolean		
Description	Car는 자동차보험의 구체적인 피보험체로 차량 고유의 속성과 운전자 조건을 가지므로 Insured를 상속하여 독립 클래스로 추출했다.	
Attributes	속성은 두 그룹으로 나뉜다. 차량 식별 정보(carId, carNumber, owner, purpose)는 차량을 고유하게 식별하고 용도를 구분하기 위해 필요하다. 차량 상세 정보(model, driver, driverScope)는 보험료 산출과 운전자 한정 조건 적용에 필요한 정보를 담는다.	
Field	Type	Description
carId	String	차량 고유의 ID
carNumber	String	차량 번호(64마0866)
driver	String	운전자
driverScope	DriverScope	운전자 범위(가족한정/본인)
model	Model	차량 모델
owner	String	차량 소유자
purpose	enum	운전 목적(개인용/업무용/영업용)
Operation	isDriverAllowed(int)는 운전자의 나이를 받아 현재 설정된 운전자 범위 조건을 충족하는지를 판단한다. 보상 처리 시 운전자 한정 위반 여부를 확인하는 데 활용된다.	
Return type	Method	Description

boolean	isDriverAllowed	운전자 범위 중 최소 연령의 운전 가능 여부를 출력하는 메서드
---------	-----------------	------------------------------------

Model

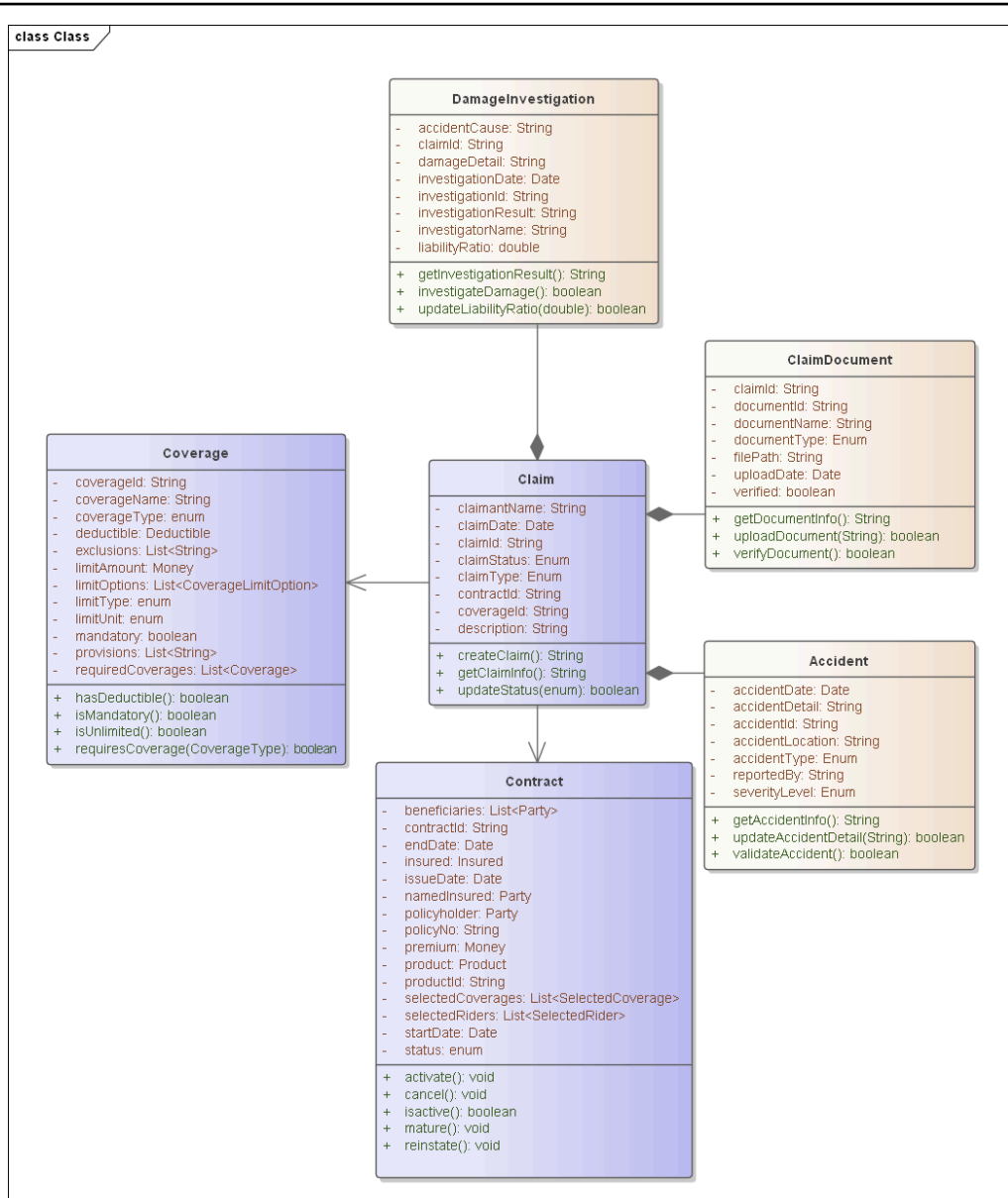
class Class Model - engineCC: int - fuelType: enum - manufacturer: String - modelId: String - modelName: String - modelType: enum - modelYear: Date		
Description	차량 모델 정보는 여러 차량이 공통으로 참조하는 마스터 데이터다. 모델 정보를 Car 안에 직접 넣으면 동일 모델을 가진 차량마다 중복 데이터가 발생하므로 독립 클래스로 추출했다.	
Attributes	modelId와 modelName은 모델을 식별하기 위한 정보다. manufacturer와 modelType은 제조사와 차종을 구분하고 modelYear는 연식을 저장한다. engineCC와 fuelType은 보험료 산출 시 차량 등급 분류에 활용되는 정보다.	
Field	Type	Description
modelId	String	엔진 cc 단위
modelName	String	차량 번호(64마0866)
modelType	enum	차종(SUV, VAN, TRUCK)
modelYear	Date	차량 연식
manufacturer	String	제조사
engineCC	int	엔진 cc
fuelType	enum	연료 종류(가솔린, LPG, 전기차)

DriveScope

class Class DriverScope - minAge: int - scopeType: enum + allowsAge(int): boolean + getMinAge(): int + getScopeType(): ScopeType + isRestricted(): boolean		
Description	DriverScope는 자동차 보험 계약 시 설정된 운전자 범위 조건을 관리하는 클래스이다. 최소 허용 연령과 범위 종류(가족한정/본인)를 저장하며, 사고 발생 시 실제 운전자가 보험 적용 대상인지 검증하는 기준이 된다.	
Attributes	속성은 두 그룹으로 나뉜다. 범위 조건 (scopeType)은 운전자를 본인으로 한정할지 가족까지 허용할지를 구분한다. 연령 조건 (minAge)은 보험 적용 가능한 최소 운전자 연령을 저장하며, 연령 미달 운전자의 사고 발생 시 보상 제외 판단의 근거가 된다.	
Field	Type	Description
minAge	int	최소 허용 운전 연령
scopeType	enum	운전자 범위(본인한정, 가족한정, 자녀)

Operation	allowsAge(int)는 입력된 나이가 연령 한정 조건을 충족하는지를 판단한다. isRestricted()는 운전자 한정 특약 가입 여부를 반환한다. getMinAge()와 getScopeType()은 외부에서 조건 값을 조회할 수 있도록 제공한다.	
Return type	Method	Description
boolean	allowsAge	연령 한정 조건을 확인하는 메서드
int	getMinAge	운전자 최소연령을 출력하는 메서드
ScopeType	getScopeType	운전자 범위를 확인하는 메서드
boolean	isRestricted	한정 특약 가입 여부를 확인하는 메서드

Claim 관계



Description

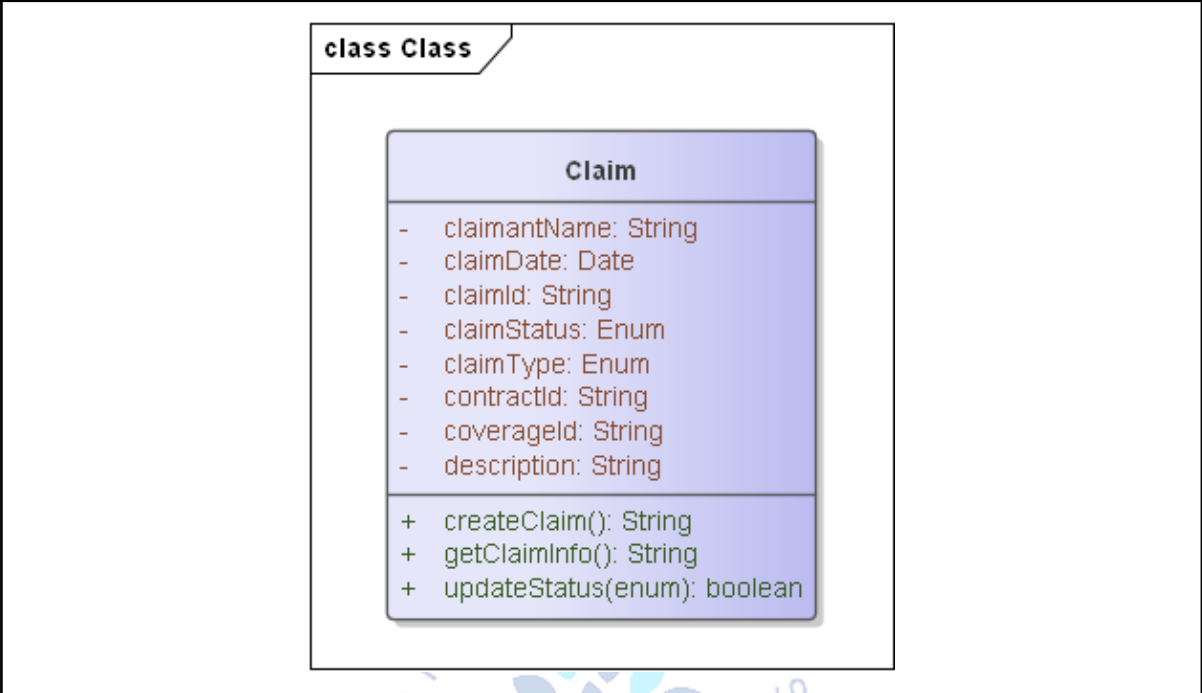
<Association>

- Claim과 Contract는 연관관계이다. Claim은 특정 보험계약을 기준으로 발생하는 청구이므로 Contract와 연결된다.
- Claim과 Coverage는 연관관계이다. Claim은 특정 담보를 대상으로 접수되는 청구이므로 Coverage와 연결된다.

<Aggregation>

- Claim과 Accident는 집합관계이다. 하나의 Claim은 사고 정보를 포함하여 관리한다.
- Claim과 ClaimDocument는 집합관계이다. 하나의 Claim은 청구 관련 증빙서류를 포함하여 관리한다.
- Claim과 DamageInvestigation는 집합관계이다. 하나의 Claim은 손해조사 정보를 포함하여 관리하며, 이를 바탕으로 후속 보상 절차가 진행된다.

Claim

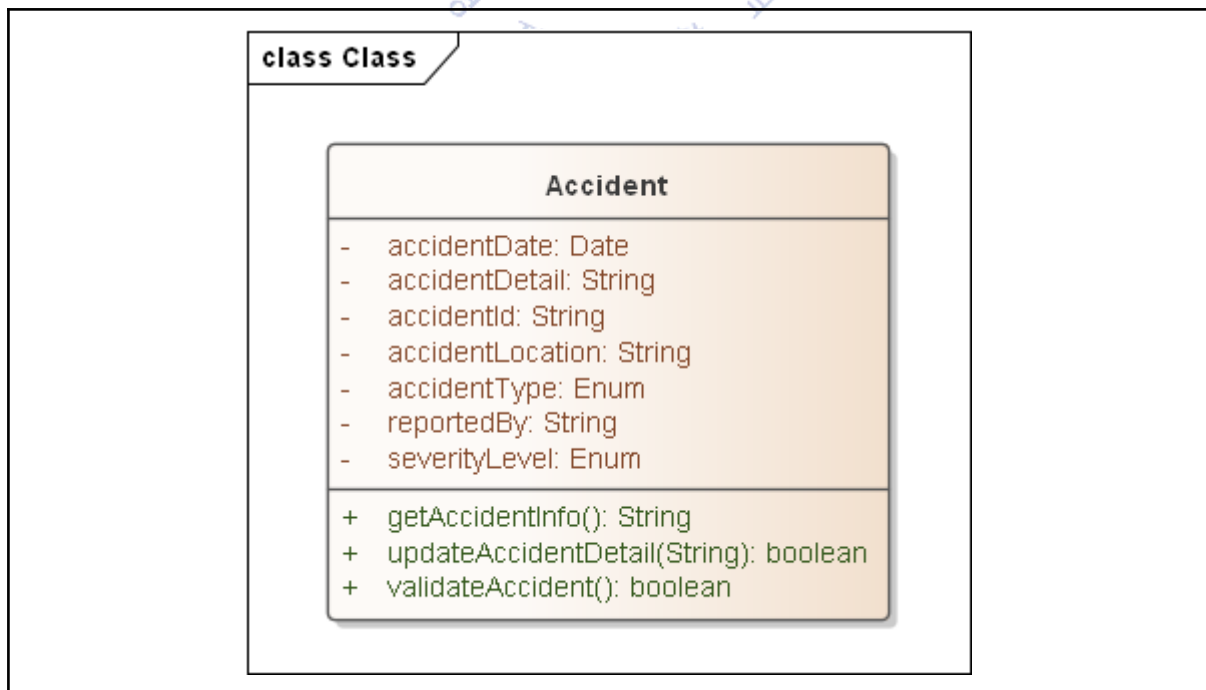


Description	Claim은 고객이 자동차 사고 발생 후 보험금을 청구한 내역을 관리하는 클래스이다. 청구일자, 청구상태, 청구유형, 담보 정보를 저장하며, 손해조사 및 보상 처리의 중심 역할을 한다. 하나의 Claim은 특정 계약(Contract)과 담보(Coverage)를 기준으로 생성되며, 사고 정보(Accident), 증빙서류(ClaimDocument), 손해조사(DamageInvestigation)같이 보상 처리에 필요한 모든 정보를 포함하여 관리한다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (claimId, contractId, coverageId)는 해당 청구건을 계약 및 담보와 연결하기 위해 필요하다. 청구 기본 정보 (claimantName, claimDate, claimType, description)는 누가, 언제, 어떤 유형으로 청구를 접수했는지와 청구 내용을 기록하며 손해조사 및 보상 처리의 근거가 된다. 진행 상태 정보 (claimStatus)는 청구가 현재 어느 단계에 있는지를 나타내며 보상 처리 전 과정의 흐름 추적 기준이 된다.	
Field	Type	Description
claimId	String	청구 ID
contractId	String	계약 ID
coverageId	String	담보 ID
claimDate	Date	청구 접수일
claimStatus	enum	청구 상태(접수완료, 심사중, 조사중, 산정 완료, 지급완료, 종결)

description	String	청구 내용
claimantName	String	청구자 이름
claimType	enum	청구 유형(대인/대물/자동차상해/자기차량)

Operation	Claim의 메서드는 보험금 청구의 생성과 상태 관리를 담당하도록 설계했다. createClaim()은 고객의 사고 접수 정보를 바탕으로 새로운 청구 건을 생성하고 고유한 claimId를 반환한다. updateStatus(claimStatus)는 심사, 조사, 산정, 지급처럼 각 처리 단계가 완료될 때마다 claimStatus를 갱신한다. getClaimInfo()는 현재 청구 건의 전체 정보를 반환하여 손해조사(DamageInvestigation) 및 손해액 산정(DamageAssessment) 단계에서 참조할 수 있도록 한다. 이 세 메서드가 청구 데이터 변경의 유일한 진입점이 되어 외부에서 청구 상태를 직접 조작하는 것을 막고 청구 처리 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
String	createClaim	청구를 생성하고 청구 ID 반환
boolean	updateStatus	청구 상태 변경
String	getClaimInfo	청구 정보 출력

Accident



Description	Accident는 보험금 청구의 원인이 되는 사고 정보를 관리하는 클래스이다. 사고 발생일, 사고 유형, 사고 위치, 신고자, 사고 상세내용을 저장하여 사고 사실을 기록한다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (accidentId)는 해당 사고 건을 고유하게 식별하기 위해 필요하다. 사고 기본 정보 (accidentDate, accidentLocation, accidentType, accidentDetail)는 사고가 언제, 어디서, 어떤 유형으로 발생했는지와 상세 내용을 기록하며 손해조사 및 보상 처리의 핵심 근거가 된다. 접수 정보 (reportedBy, severityLevel)는 사고를 신고한 주체와 피해 규모를 나타내며 보상 우선순위 및 담당자 배정의 기준이 된다.	
Field	Type	Description
accidentId	String	사고 ID
accidentDate	Date	사고 발생일
accidentLocation	String	사고 발생 위치
accidentType	enum	사고 유형(충돌, 접촉, 화재, 도난, 전복)
accidentDetail	String	사고 상세 내용
reportedBy	String	사고 신고자
severityLevel	enum	사고 심각도(경미, 보통, 중대)

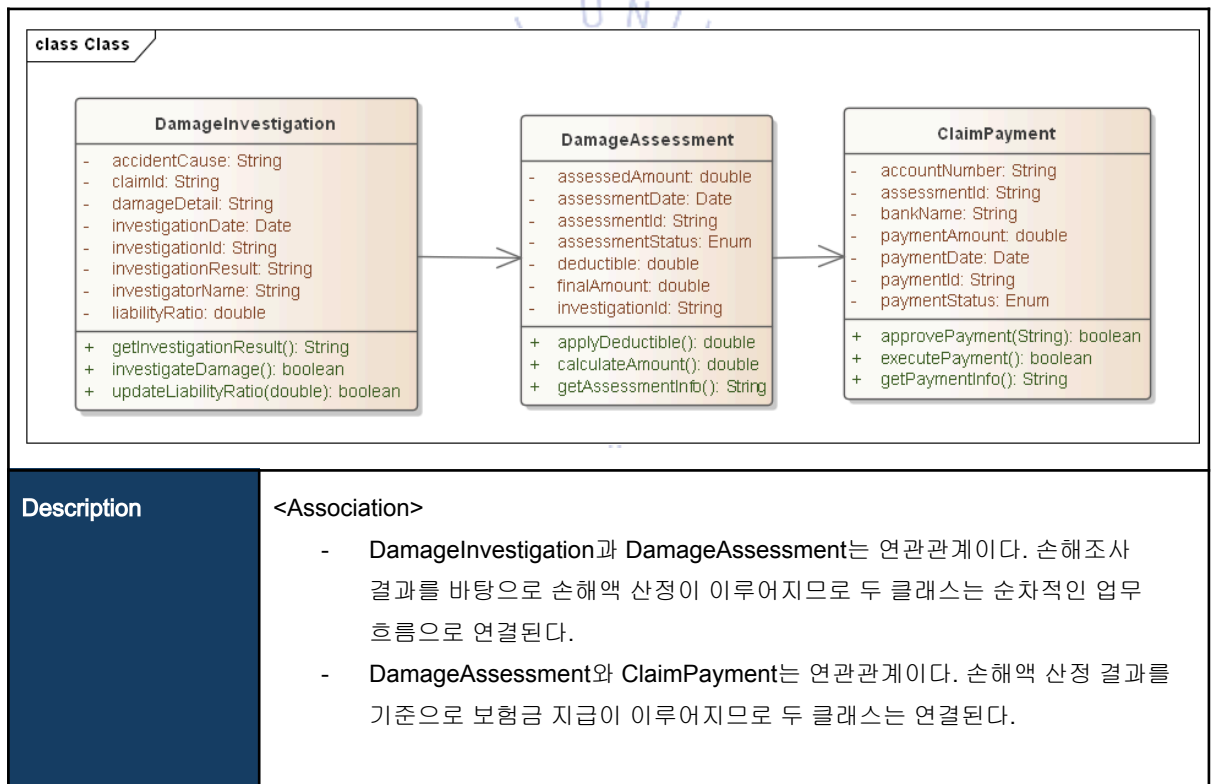
Operation	Accident의 메서드는 사고 정보의 조회, 수정, 유효성 검증을 담당하도록 설계했다. getAccidentInfo()는 사고의 전체 정보를 반환하여 손해조사(DamageInvestigation) 및 청구(Claim) 단계에서 참조할 수 있도록 하고, updateAccidentDetail(String)은 초기 접수 이후 추가로 확인된 사고 상세 내용을 갱신한다. validateAccident()는 입력된 사고 정보가 보험 약관상 보상 가능한 유형인지 유효성을 검증한다. 이 세 메서드가 사고 데이터 변경의 유일한 진입점이 되어 외부에서 사고 정보를 직접 조작하는 것을 막고 사고 관리 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
String	getAccidentInfo	사고 정보 출력
boolean	updateAccidentDetail	사고 상세 정보 수정
boolean	validateAccident	사고 정보 유효성 확인

ClaimDocument

class Class ClaimDocument - claimId: String - documentId: String - documentName: String - documentType: Enum - filePath: String - uploadDate: Date - verified: boolean + getDocumentInfo(): String + uploadDocument(String): boolean + verifyDocument(): boolean		
Description	ClaimDocument는 청구 처리에 필요한 증빙서류를 관리하는 클래스이다. 서류명, 서류유형, 파일경로, 업로드일자, 검증여부를 저장한다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (documentId, claimId)는 해당 서류와 보험금 청구 건을 연결하기 위해 필요하다. 서류 기본 정보 (documentName, documentType, filePath)는 서류의 이름, 종류, 실제 저장 위치를 기록하며 서류 조회 및 검토의 근거가 된다. 처리 상태 정보 (uploadDate, verified)는 서류가 언제 업로드되었고 검증이 완료되었는지를 나타내며 청구 심사 진행 상황 추적의 기준이 된다.	
Field	Type	Description
documentId	String	서류 ID
claimId	String	청구 ID
documentName	String	서류명
documentType	enum	서류 유형(사고경위서, 진단서, 견적서, 사진자료)
filePath	String	파일 저장 경로
uploadDate	Date	업로드 일자
verified	boolean	검증 여부
Operation	ClaimDocument의 메서드는 청구 서류의 등록과 검증 처리를 담당하도록 설계했다.	

	uploadDocument(String path)는 고객이 제출한 서류를 바탕으로 문서를 시스템에 등록하고 filePath와 uploadDate를 저장한다. verifyDocument()는 업로드된 서류의 유효성을 검토하여 verified 상태를 갱신한다. getDocumentInfo()는 해당 서류의 상세 정보를 반환하여 청구 심사 단계에서 활용될 수 있도록 한다. 이 세 메서드가 서류 처리의 유일한 진입점이 되어 외부에서 검증 상태를 직접 조작하는 것을 막고 서류 관리 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
boolean	uploadDocument	청구 서류 업로드
boolean	verifyDocument	서류 검증
String	getDocumentInfo	서류 정보 출력

DamageInvestigation 관계



DamageInvestigation

class Class DamageInvestigation - accidentCause: String - claimId: String - damageDetail: String - investigationDate: Date - investigationId: String - investigationResult: String - investigatorName: String - liabilityRatio: double + getInvestigationResult(): String + investigateDamage(): boolean + updateLiabilityRatio(double): boolean		
Description	DamageInvestigation은 사고로 인해 발생한 손해를 조사하는 클래스이다. 조사담당자, 조사 일자, 사고 원인, 손해 내용, 과실 비율, 조사 결과를 기록하여 손해액 산정의 기초 자료를 제공한다. 하나의 DamageInvestigation은 특정 보험금 청구(Claim)에 종속되어 생성되며, 조사 완료 후 산출된 손해 정보는 손해액 산정(DamageAssessment)으로 전달되어 실제 지급 보험금 결정의 근거가 된다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (investigationId, claimId)는 해당 손해조사 건과 보험금 청구 건을 연결하기 위해 필요하다. 조사 기본 정보 (investigatorName, investigationDate, accidentCause)는 누가, 언제, 어떤 사고 원인으로 조사를 수행했는지를 기록한다. 조사 결과 정보 (damageDetail, liabilityRatio, investigationResult)는 실제 손해 내용, 과실 비율, 최종 조사 결론을 저장하고 손해액 산정(DamageAssessment)의 핵심 입력값이 된다.	
Field	Type	Description
investigationId	String	손해조사 ID
claimId	String	청구 ID
investigatorName	String	조사 담당자명
investigationDate	Date	조사 일자
accidentCause	String	사고 원인
damageDetail	String	손해 조사 내용

liabilityRatio	double	과실 비율
investigationResult	String	조사 결과

Operation	DamageInvestigation의 메서드는 손해조사의 수행과 결과 관리를 담당하게 설계했다. investigateDamage()는 사고 현장 및 손해 내용을 조사하여 조사 결과(damageDetail, accidentCause)를 기록하고 updateLiabilityRatio(double)는 조사 결과를 바탕으로 과실 비율(liabilityRatio)을 산정하여 갱신한다. getInvestigationResult()는 조사 완료 후 최종 결과를 반환하여 손해액 산정 단계로 전달한다. 이 세 메서드가 조사 데이터 변경의 유일한 진입점이 되어 외부에서 조사 결과를 직접 조작하는 것을 막고 조사 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
boolean	investigateDamage	손해 조사 수행
String	getInvestigationResult	손해 조사 결과 출력
boolean	updateLiabilityRatio	과실 비율 수정

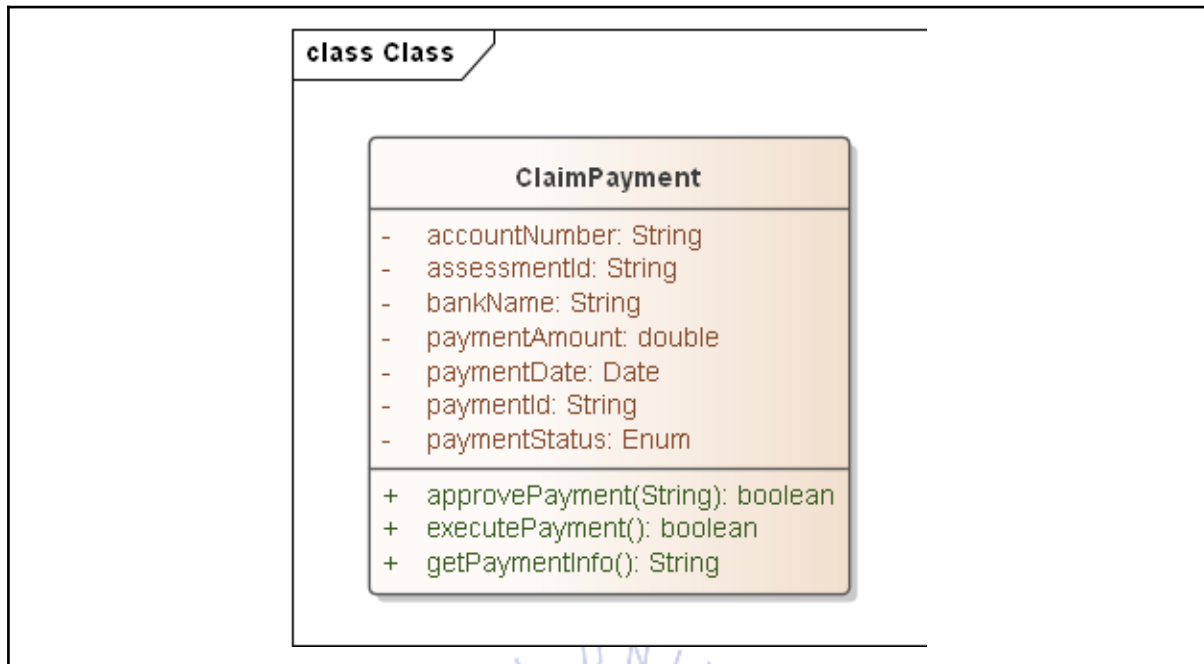
DamageAssessment

class Class DamageAssessment - assessedAmount: double - assessmentDate: Date - assessmentId: String - assessmentStatus: Enum - deductible: double - finalAmount: double - investigationId: String + applyDeductible(): double + calculateAmount(): double + getAssessmentInfo(): String	
Description	DamageAssessment는 손해조사 결과를 바탕으로 최종 손해액을 산정하는 클래스이다. 산정금액, 자기부담금, 최종 지급예정금액, 산정상태를 저장하며 실제 지급 보험금의

	기준을 결정한다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (assessmentId , investigationId)는 해당 산정건과 손해조사 결과를 연결하기 위해 필요하다. 금액 정보 (assessedAmount , deductible , finalAmount)는 손해조사를 통해 산출된 원래 손해액, 자기부담금 공제 전후의 금액을 단계적으로 저장하며 실제 지급 보험금 결정의 근거가 된다. 진행 정보 (assessmentDate , assessmentStatus)는 산정이 언제, 어떤 상태로 처리되고 있는지를 나타내며 업무 진행 상황 추적의 기준이 된다.	
Field	Type	Description
assessmentId	String	손해액 산정 ID
investigationId	String	손해조사 ID
assessedAmount	double	산정 금액
deductible	double	자기부담금
finalAmount	double	최종 지급 예정 금액
assessmentDate	Date	산정 일자
assessmentStatus	enum	산정 상태(산정전, 산정중, 산정 완료, 승인완료)

Operation	DamageAssessment의 메서드는 손해액 산정의 단계별 계산 로직을 캡슐화하도록 설계했다. calculateAmount()는 손해조사 결과를 바탕으로 최초 손해액(assessedAmount)을 계산하고 applyDeductible()은 내부적으로 산정된 금액에서 자기부담금을 공제하여 최종 지급 예정 금액(finalAmount)을 확정한다. getAssessmentInfo()는 현재 산정 건의 상태와 금액 정보를 반환한다. 이 세 메서드가 금액 산정의 유일한 진입점이 되어 외부에서 금액 값을 직접 조작하는 것을 막고 산정 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
double	calculateAmount	손해액 계산
double	applyDeductible	자기부담금 반영
String	getAssessmentInfo	손해액 산정 정보 출력

ClaimPayment



Description	ClaimPayment는 손해액 산정 결과에 따라 보험금을 지급하는 클래스로 지급 정보를 관리한다. 지급금액, 지급일자, 지급상태, 은행명, 계좌번호, 결제 날짜를 저장하며 보험금 지급 승인과 실행을 담당한다.	
Attributes	속성은 세 그룹으로 나뉜다. 식별 정보 (paymentId, assessmentId)는 해당 지급 건과 손해액 산정 결과를 연결하기 위해 필요하다. 지급 정보 (paymentAmount, paymentDate, paymentStatus)는 실제 보험금이 얼마나, 언제, 어떤 상태로 처리되는지를 나타내며 지급 진행 상황 추적의 근거가 된다. 계좌 정보 (bankName, accountNumber)는 보험금을 실제로 송금하기 위한 수취인 계좌 정보를 저장한다.	
Field	Type	Description
paymentId	String	지급 ID
assessmentId	String	손해액 산정 ID
paymentAmount	double	지급 금액
paymentDate	Date	지급 일자
paymentStatus	enum	지급 상태(지급대기, 승인완료, 지급완료, 지급실패)
bankName	String	지급 은행명
accountNumber	String	지급 계좌번호

Operation	ClaimPayment의 메서드는 보험금 지급 프로세스의 단계별 처리를 표현하도록 설계했다. approvePayment(String)는 손해액 산정 ID를 받아서 손해액 산정 결과를 검토하고 지급 승인 여부를 결정한다. executePayment()는 승인된 보험금을 실제 계좌로 송금하는 실행을 담당한다. getPaymentInfo()는 현재 지급 건의 상태와 상세 정보를 반환한다. 이 세 메서드가 지급 상태 변경의 유일한 진입점이 되어 외부에서 paymentStatus를 직접 조작하는 것을 막고 상태 전이 규칙을 클래스 안에 캡슐화한다.	
Return type	Method	Description
boolean	approvePayment	보험금 지급 승인
boolean	executePayment	보험금 지급 실행
String	getPaymentInfo	지급 정보 출력

